

MSc thesis in Geomatics

Spatio-temporal Transformers for Wildfire Prediction

Carlo Cordes

June 2026

A thesis submitted to the Delft University of Technology in partial fulfillment of the requirements for the degree of Master of Science in Geomatics

Carlo Cordes: *Spatio-temporal Transformers for Wildfire Prediction* (2026)

© This work is licensed under a Creative Commons Attribution 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by/4.0/>.

The work in this thesis was carried out in the:



Geo-Database Management Centre
Delft University of Technology

Supervisors: Dr. Azarakhsh Rafiee
Justin Schembri

Co-reader: Dr. Simona Bianchi

Contents

1. Introduction	2
1.1. Context	2
1.2. Remote Sensing in Wildfire Modelling	2
1.3. Vision Transformers for Wildfire Prediction	3
1.4. Problem Statement	3
2. Related Work	4
2.1. Drivers & Metrics of Wildfire Risk Modelling	4
2.2. The Transformer	5
2.3. Multimodal Learning Tasks in Remote Sensing	9
2.4. Memory Acceleration	10
2.5. Research Gap	10
3. Data	12
3.1. Data Acquisition	12
3.2. Feature Engineering	13
3.3. Processing	15
4. Methodology	18
4.1. Workflow & System Overview	18
4.2. Dataset Construction	18
4.3. Model Architecture	20
4.4. Training Routine	27
4.5. Experiments	30
5. Results	32
5.1. Experiment 1: The Baseline Model	32
5.2. Experiment 2: Sample Extent	34
5.3. Experiment 3: Lead-Time	36
5.4. Experiment 4: Ablation Studies	38
6. Conclusions	42
Bibliography	44

TODOs

TODO 1	Work in progress	10
TODO 2	Add UML figure of GG	13
TODO 3	Figure of zarr slicing and union	14
TODO 4	Figure of 3 step process here?	14
TODO 5	work in progress	18
TODO 6	Update figure of timestamp configurations	18
TODO 7	Complete model architecture diagram and reference	20
TODO 8	Figure of reflective patch padding	20
TODO 9	Finalization in progress, left blank here	42

1. Introduction

1.1. Context

Perhaps one of the most impactful emerging challenges is the increasing threat of natural disasters, in particular wildfires. The California wildfires of 2025 represent the costliest natural disaster in the history of the United States with some estimates reaching 275 Billion USD. The biggest contributors were the Palisades and Eaton fires with events igniting almost simultaneously, making emergency resource management and mitigation measures extremely difficult to coordinate. Experts argue that a combination of neglected vegetation management, strong prevailing winds and forest health issues were at the core of an event that ultimately destroyed 9594 hectares and over 6800 structures (Babrauskas, 2025). Between 2024 and 2025, total Carbon emissions related to fire rose 9% above the yearly average. As temperature averages reach new maxima year after year, the correlated frequency of wildfires shift with it. In Southern California, estimates quantify the total burnt area to be as much as 25 times larger as a direct consequence of the changing climate. In Northeast Amazonia, extreme fire weather is up to 70 times more likely as a result (Kelley, 2025).

1.2. Remote Sensing in Wildfire Modelling

A great deal of scientific effort has been invested to make preparedness for wildfire risk a data-driven discipline. While historically experts relied on environmental assumptions (Rothermel, 1972) to gain knowledge on environmental conditions, we today have an abundance of data freely available and powerful computational methods to analyze them. With satellite imagery being publically distributed in accessible formats, physical phenomena like natural hazards may be detected, tracked and characterized from afar without expert knowledge of the on-site conditions.

The spatial and temporal coverage of satellite data coincides well with the threats posed by natural hazards as these are highly stochastic and globally relevant. While local knowledge used to be required to characterize fire fuel conditions, these can in todays time be monitored from afar. Deep learning methods in particular require large datasets for training, making satellite-collected remote sensing data the ideal discipline that has found great success in a variety of applications.

Since applying ML in the wildfire realm, the focus has shifted from guiding mitigation via reaction to also using prediction as a counter measure. The difficulty with providing valuable wildfire risk quantifications arises directly from heterogeneity of the data underlying these effects. The combination of static images (e.g. topography, land use) and dynamic variables (e.g. NDVI, land surface temperature).

Approaches, amongst others, included deep neural networks and long-short-term memory setups, which lied on sequential nature of timer-series data to capture their information about physical processes (Durlević et al., 2025). It is however exactly this sequentialism which poses as the mainbottleneck. These networks inherently, while being able to memorize and learn short-term information, the longer the input sequences to be predicted are, the worse the predictions become. The same effect is apparent for convolutions, which characterize convolutional neural netoworks. These excell at capturing local features, however struggle distant

1.2. Remote Sensing in Wildfire Modelling

relationships. Being able to capture temporal and spatial relationships is crucial for wildfire modeling.

1.3. Vision Transformers for Wildfire Prediction

Transformers address this issue as the data processed is permutation invariant as it is inherently encoded in each input token (Vaswani, 2017). Having proven to be effective in both image classification (Dosovitskiy et al., 2020) and time-series forecasting (Li, 2021), it is apparent that transformers cater perfectly to the problem setup of wildfire prediction, as it can learn an understanding of deep physical processes as well as the inherent meaning of space-time. Furthermore, transformers cater precisely to the latent space blind spot by sequentially-based architectures, as they operate entirely in parallel. Additionally to being able to explicitly encode spatial dimensions in transformer models, time can be handled analogously, alleviating the issue of vanishing information in long sequences. Finally, the transformer handles its computations in parallel, making it highly scalable.

Much research has been done into integrating such spatio-temporal transformers in wildfire related topics, especially for predicting wildfire spread (Jain et al., 2020). The high susceptibility of an ignited fire to wind, terrain and drought indices have proven the spatio-temporal setup to be quite useful in predicting spread directions, thus serving public authorities as a valuable tool.

1.4. Problem Statement

Despite being an established method for predictive methods in a variety of sectors, one of which being wildfire prediction, little work has been done into treating wildfire occurrence separately from risk of ignition and its consequent spread. The spatio-temporal transformer leverages a highly parallelizable, efficient framework, with the ability to learn a variety of tasks. With the ability to process and denote explicitly the spatial and temporal dependencies of data modularities in combination with spatial attention blocks, the wide availability of multi-modal satellite becomes an asset. The approach of this investigation relies on fusion of both static and dynamic environmental datasets, as well as explicit infusion of spatial and temporal embedding of space-time. Together with historical records of burnt area as ground truth, we form a predictive pipeline between environmental data and reality which specifically targets the generalized risk of wildfire occurrence.

This report is structured as follows: Section 2 reviews the most important related work into drivers of wildfire, an introduction to the spatio-temporal transformer architecture, as well as existing studies applying it to wildfire scenarios. Most importantly, the research questions guiding this work and experiments that make up the investigation are explained. In Section 4 an account of the methodology used to construct datasets, the overall structure of training as well as details regarding the proposed spatio-temporal transformer model architecture are given. Results of all experiments as well as a discussion into all research questions and experiments is presented in Section 5 before a concise summary in Section 6.

2. Related Work

2.1. Drivers & Metrics of Wildfire Risk Modelling

Wildfire events are highly complex phenomena. First and foremost, due to their highly violent nature, fires in the wild as well as in urban areas pose as one of the most destructive of natural hazards. Furthermore, the stochastic yet extremely rare nature of such events pose as a challenge in quantifying, reasoning and mitigating such events. On top of being difficult to predict, such fire events are highly locally concentrated. In 25 years of recorded data in Spain, a region of 6% of the total landmass is responsible for the majority of outbreaks (Álvarez-Díaz et al., 2015). Having a close pulse on such events is crucial as forest systems serve a crucial importance for establishing a healthy carbon balance and are a fundamental component to the mitigation of global carbon combustion (Illarionova et al., 2022).

In recent decades, many areas world-wide have been experiencing an increase of such events. While a great deal of the scientific discussion has been centered around correlating land use and risk mitigation to the occurrence of these fires, a large factor can be attributed to a novel set of climate conditions. In the United States, more particularly in the western state of California, a large spike in such events from the 1980s and onwards can be found to correlate significantly with the conditions a warming climate poses (Westerling et al., 2006). With elongated dry summers, drought and stagnating vegetation health, especially regions of mid-elevation are found to be directly be affected by the characteristics of a warming climate.

Assessing and describing the effects of wildfires can be essentially split into two disciplines. Conceptually, a fire disaster describes the propagation of a fire event through combustible material, given the initial start of a fire. The terminology to focus on here are the probability of ignition of a fire and the thereby causal spread of it. Describing these two disciplines separately as well as quantifying their interdependencies lies at the heart of quantifying the risk a fire poses (Castro et al., 2010). In turn, this allows for authorities to make concentrated choices regarding mitigation measures, in which both spark and ignition have to be addressed. In essence, the discipline can be contextualized as the conditional probability of a spread of a fire given an ignition.

Studies find global wildfire ignition with 90% likelihood to be caused by human activity (Arndt et al., 2013). A further investigation of fire ignitions in California, US used ten years' worth of ignition data to show the local dispersion of events caused by humans versus other effects. It showed strong spatial between human-induced events and urban areas as well as remote nature and lightning strikes (Chen & Jin, 2022). It could be shown that this effect is also highly locally optimized and needs to be assessed regionally, once again underlining the importance of good interpretation of local climate health conditions.

Taking ignition probability out of the picture, there are a few crucial factors influencing the extent, intensity and probability of spread. Most importantly, average daily temperatures, correlating with summer season make the biggest effects. Along with a deprecated annual rainfall rate and sparsity thereof, the general distribution of anthropogenic activity is a notable factor. Furthermore, studies have linked gross domestic product (GDP) density to the extent of fire events (Aldersley et al., 2011). Urban areas therefore not only contain more sealed surfaces,

2.1. Drivers & Metrics of Wildfire Risk Modelling

posing as a natural fire barrier but with rising GDP per area, extended mitigation measures could be the underlying factor of less burnt area occurring in such high value producing areas.

Other factors influencing burn extent include natural difference vegetation indices (NDVI). This monitors vegetation health through infrared radiation from satellite stations. Vapor pressure, humidity are contributing factors. A factor that can lead to containment or further acceleration of such events is topography, as terrain is more susceptible to propagating fire in high-slope conditions. Similarly, wind has the analogous ability to shift flame angle to set neighboring combustible material afire. Lastly, determining biomass density and fuel moisture are avid determinants of risk (Rafaqat et al., 2022). Wind is an especially dangerous factor as at high velocities it can also lead to transport of burning materials, sparking new fires elsewhere, a factor that was especially harmful in the January 2025 fires in Los Angeles (Syphard et al., 2022).

2.2. The Transformer

2.2.1. Background & Motivation

Processing data in the deep learning domain, whether text or image, was historically dominated by Recurrent Neural Networks (RNN) or Long-Short-Term Memory (LSTM). These types of learning networks rely on a chain-like structure, processing data in series to make a prediction. Speaking specifically for processing sequences of satellite imagery, a bottleneck arises when scaling to large datasets. As the processing of data sample n_t is dependent on n_{t-1} , such a process is inherently inefficient due to its incompatibility to be parallelized. Furthermore, such models are subject to the *Vanishing Gradient* effect. This describes the loss of information a sequential model encounters as the size of the input sequence increases. Every element is dependent on the one before it, so as a signal passes through larger amounts of computations, information will get lost (Hochreiter, 1998).

A fundamental breakthrough was made in modern technology with the transformer architecture, the goal of which was to eliminate the sequential computation bottleneck. In direct consequence, this would tackle the the issue modelling accuracy being dependent on the distance of elements of the encoded input sequence. The transformer, as originally conceived for neural translation tasks, was the pioneering idea in this regard. The proposed architecture relies not on processing data sequentially but treating the input data entirely at once. Inputs no longer rely on their relative position in the sequence but are much rather position invariant concepts. Every part has the direct ability to interact with all others without relying on those in between (Vaswani, 2017). The regression analysis was consequently replaced by the attention mechanism, that allows sequence segments to self-establish their relevance and correlation to each other.

2.2.2. Architecture

The core idea behind the transformer is to form predictions given an input, depending on the application. As mentioned above, the initial design involved translating text between languages not just word for word, but with strong relational semantic meaning. This same concept was later adapted for a variety of different tasks, perhaps most prominently text generation tasks. Modern chatbots use this very mechanism to form educated prediction of outputs given a user input sequence.

Generative Transformers that produce either images or text from input sequences are pre-trained, meaning their weights are set. Under the hood are various matrix multiplication processes that turn inputs into outputs. Crucial for this to work however is the processing of

input sequences into a structure the transformer architecture can understand. To do this, we consider partitions of the input as so-called *tokens* (Naeve et al., 2024) as shown in Figure 2.1.

Raw: The capital of Italy is
Tokens: [The] [capital] [of] [Italy] [is]

Figure 2.1: Tokenization of an input

Each of the input tokens is then processed via a process called embedding. This simply assigns every token to a high dimensional vector:

$$\text{Italy} \rightarrow \vec{e} = \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ \vdots \end{pmatrix} \in \mathbb{R}^{d_{\text{embed}}} \quad (2.1)$$

To an untrained model this might appear like a random projection of a word into a vector, however this process is learnable, meaning that the way a token might be transformed into an embedding is optimized as a model trains. As a result, trained models gather their entire understanding of reality by assigning many tokens each a distinct position in a very high dimensional embedding vector space. A trained model not only has a position for every term in its vocabulary, but the relative position of these can directly embody similarities of terms in plain language. For example, consider a pre-trained transformer that holds embeddings for the terms *brother*, *sister*, *aunt* and *uncle* in Figure 2.2.

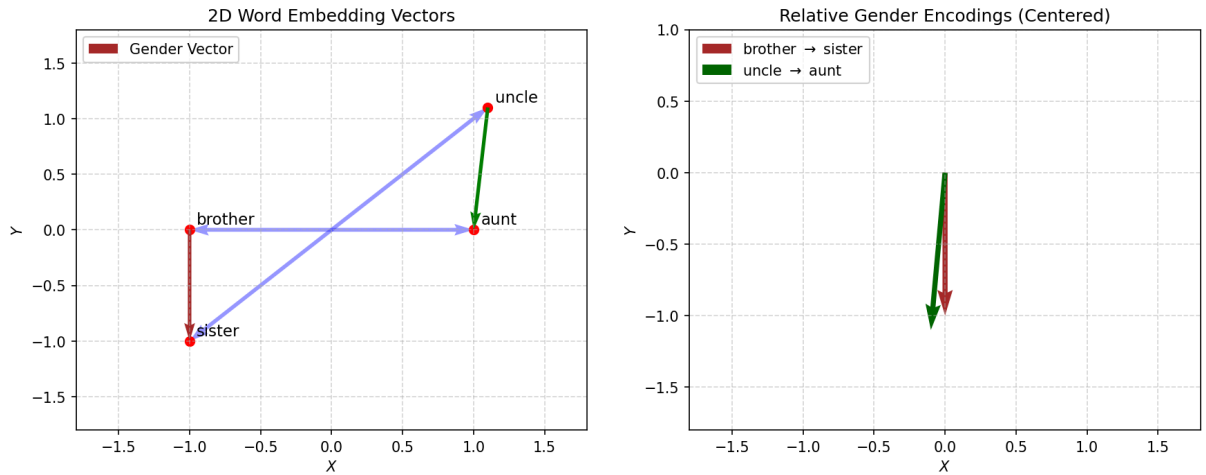


Figure 2.2: Organization of tokens into vector space and relative positional meanings

The important notion to realize is that a trained transformer is able to relate certain terms to one another. Subtracting the vector of *aunt* and *uncle* will yield a similar vector to the one when applying this logic to *brother* and *sister*, a vector that to the transformer encodes a logic of relative gender. Furthermore, taking the dot product of two vectors will provide information about similarity of these two vectors. Analogously to vector geometry, the dot product of two unit vectors pointing in different directions will yield zero:

$$\hat{e}_i \cdot \hat{e}_j = \delta_{ij} = \{1 \text{ if } i = j, \text{ else } 0 \quad (2.2)$$

Suppose such vectors were describing directions in a coordinate system. A zero dot-product dictates that these two vectors are very dissimilar, as they encode different directions. The

2.2. The Transformer

same applies to the embedding dimension. A large dot product of two embeddings signifies similarity between these terms (Sanderson, 2024a).

2.2.3. Attention

Embedding the tokens of an input sequence like in Figure 2.1, for now only gives a vectoral meaning to each individual one, rather than as a complete input sequence. However, as the core concept here is to make predictions from an input sequence, a mechanism relating the meaning of each token into a conglomerate understanding is necessary. This exact problem is solved by the concept of attention, a powerful matrix encoder that not only serves as the basis of this project but is responsible for the immense growth in the artificial intelligence sector (Sanderson, 2024b).

Despite being powerful, this concept is very simple. It determines the relative importance of all embeddings to all other embeddings in the input and subsequently updates each embedding, depending on this relative importance. This step can be conceptualized as tokens absorbing information from others. For example, the embeddings of *green apple* will hold much richer meaning after an attention block as the token *apple* will have **attended** to *green*. Thereafter, the embedding of apple is updated. It now sits in a slightly shifted position in embedding space, carrying this richer, updated information.

Mathematically speaking, the attention block is a scaled matrix multiplication. To apply attention to the input sequence, first let the concatenation operation of all tokens of a sequence into a structure that will be called the Value matrix V :

$$[\text{The}] [\text{capital}] [\text{of}] [\text{Italy}] [\text{is}] \rightarrow \begin{pmatrix} \vdots & \vdots & \vdots & \vdots & \vdots \\ \vec{E}_1 & \vec{E}_2 & \vec{E}_3 & \vec{E}_4 & \vec{E}_5 \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix} = V \quad (2.3)$$

For every embedding E_i a query vector (Q_i) and a key vector (K_i) is created via convolution of weighted key and query matrices that are convolved with the embeddings of each token:

$$Q_i = W_Q \cdot \vec{E}_i \quad (2.4)$$

$$K_i = W_K \cdot \vec{E}_i \quad (2.5)$$

Attention is then formulated as scaled the matrix multiplication of these two matrices, using SoftMax to scale the outputs to the domain $[0, 1]$ and scaled by the value matrix $\vec{V}$ (Vaswani, 2017):

$$\text{Attention}(K, Q, V) = \text{SoftMax}\left(\frac{Q \cdot K^T}{\sqrt{d_{\text{embed}}}}\right) \cdot V \quad (2.6)$$

This formula acts as a bridge allowing each embedding to inform itself about the contents and relevance of all other embeddings. Every embedding possesses a query Q to which the other tokens respond with a key value K . The dot product between these two in Equation (2.6) thus defines the relevance of tokens to one another. The greater the output of this dot product, the higher the attention value will be for a token pair. The result of the scaled dot product is multiplied by the value matrix. In simple terms it can be read: one token uses its query vector to get information about another with its key vector. If that information is relevant to it and the dot product is high, a weighted part of the queried token is added onto the embedding of the querying token.

This process is called self-attention, as the input sequence is only compared to itself. The analogous process of comparing two sequences, for example in translation tasks. The magnitude to which embeddings attend to one another is entirely governed by the weights in W_k and W_q , which are initialized at random and learned during model training.

The resulting attention values are then used to update the embeddings to hold richer meaning:

$$\vec{E}'_i = \vec{E}_i + A_i \quad (2.7)$$

Attention rarely is carried out in such single operations but in many blocks. The raw embeddings are fed to each of these blocks that might contain many of these operations, individually called heads. Each block and each head is thus able to learn slightly different relational attributes between the embeddings. This is called multi-headed attention (Shaw et al., 2018).

Crucially, the computations of each head and block are independent of each other, making this process highly parallelizable and computationally convenient to compute with large Graphics Processing Units (GPU).

2.2.4. Vision Transformers

The concept of using embedded sequences as predictors quickly caught on for vision tasks. The bottleneck resided in establishing an analogy to what a token should represent in images. Using the naive approach in extracting a token for each pixel value in every image quickly scales above computable limits as using N tokens for an image of height h and width w scales quadratically with image resolution ($N = h \times w$). Furthermore, attention processes also scale as $O(n_{\text{tokens}}^2)$ as per Equation (2.6), this would quickly lead to unfeasible computations (Papa et al., 2024). The solution was found in extracting not raw pixel values, but patches of them. By flattening each patch into a vector and projecting this into an embedding dimension an analogous structure to embedding was found (Dosovitskiy et al., 2020). It is hereby important that the embedded values are not the exact pixel values but are the product of a convolution operation. This mechanism outperformed convolutional approaches on predicting labelled such as ImageNet, CIFAR-100 or BTAB.

The transformer notably solved one of the main concerns of convolutional neural network approaches, which involves using a kernel as a sliding window to extract features. This process however, while being an asset at extracting local features, failed to correlate features that might have a large extent or distant relationships in an image (Wang et al., 2025). Because transformers have the ability to self-weight relationships of features, this issue is almost entirely resolved. An investigation on the the large dataset Imagenet-21k could show that vision transformer models consistently outperform convolution approaches, amongst others due to its enhanced feature recognition (Ridnik et al., 2021).

A new challenge with the transformer architecture is that in general, compared to its predecessors, such a model requires an extensive set of pre-labelled data in order not just to learn dependencies of its input, but to generalize well to unseen data (Khan et al., 2023).

Transformers performed well with simple classification tasks, but could initially not perform well in denser tasks, image segmentation being a prime example. This is in part due to the excess of attention calculation a model runs through for one prediction. An adaptation to this was found in the sliding-window (SWIN) which particularly for large-scale images bridged the gap of computational time from text tasks to the image interpretation (Liu et al., 2021). By

2.2. The Transformer

only computing self-attention between non-overlapping local windows rather than the entire set, the approach brought greater efficiency.

An invaluable advancement to vision transformer came with frequent applications to remote sensing data. As satellite data is grid-like it is an ideal application. This coincided with a further enhancement to the pipeline: positional encodings. Here, the set of embedded image patches are pre-weighted, proportionally to their distance to other patches. This serves as a kick-starter to the later cross-patch attention process in which close patches will conduct attention more closely due to their initial similarity induced by the positional encoding (Bazi et al., 2021). Early approaches adapted rotary positional encoding from natural language processing into vision approaches (Heo et al., 2024). Further progress was made with the proposal of using overlaid sinusoidal encodings, that do not just pass absolute encoding but relative ones.

This concept could be further extended the temporal dimension. Input images with multiple steps across time could not only share weights given their relative position, but also would explicitly encode time via the same logic. Images of neighboring timesteps could receive more similar encodings while distant ones could not conduct attention as closely (Arnab et al., 2021; Xu et al., 2019).

2.3. Multimodal Learning Tasks in Remote Sensing

Spatio-temporal attention mechanism find a wide array of applications in the remote sensing sector as with sufficiently large datasets, they tend to outperform other architectures (Wang et al., 2024). Specifically for climate phenomena prediction, learning architectures performed well, even before the initiation of vision transformers. Notably, a Bayesian neural network architecture was able to produce numerical predictions of el Niño, taking a multi-modal, multi-year dataset to produce valuable prediction results. The method elegantly predicts sea-surface temperatures under varying lead-time and at different time scales (Luo et al., 2022). The same study was conducted with *Earthformer*, a new transformer architecture that specializes in predicting spatio-temporal datasets (Gao et al., 2022). By dividing datasets not just into patches, but into cuboids via two directional and one temporal dimension, the model uses so-called cuboid-attention to forecast El Niño/Southern Oscillation, outperforming earlier LSTM and convLSTM methods.

2.3.1. Wildfire Applications

With natural catastrophe modelling being heavily reliant on data, machine learning architectures arrived in this sector as well. Notably, a study focusing on Serbia studied three different deep learning approaches (XGBoost, Kolmogorov-Arnold networks and neural networks) to compare their prediction abilities. By combining a multi-modal dataset to predict historical burn-map, the deep neural network (DNN) could show the best prediction accuracy with 83.4% (Durlević et al., 2025)

As large amounts of resources in the sector are being devoted to wildfire mitigation, large academic interest is focused on predicting wildfire spread (Jain et al., 2020). Specifically for transformers this area seems to be promising. Focusing on fire spread in the North American region, a study could deliver excellent results using a Swin-based transformer mechanism to deliver next-day fire spread estimates (Li & Rad, 2024). Similarly, CNN have successfully been combined with encoder-decoder transformer mechanisms to do similar work. By specifically encoding forest fire status into the predicting dataset, better predictive accuracy was achieved. By relying on environmental markers, sociological indicators and fuel conditions, methods like FiRE-HNL use physics-infused approaches to train models. This discipline could be extended

to serve multi-day predictions derived from MODIS & ERA5 data, once again outperforming CNN-approaches with F1 scores of 0.75 (Andrianarivony, 2024; Dubey et al., 2025).

A different sub-discipline focuses on risk metrics with respect to wildfire detection, a factor specifically important for non-urban areas. Another discipline this serves is examining ignition probabilities. By being able to detect wildfires soon after ignition, the root cause of the fire can be reliably identified and used to better understand initial ignition conditions (Dong et al., 2025).

Initial approaches which model ignition and spread probability as a joint risk factor in combination with transformer models look promising. First benchmarks have shown that combining large multi-year datasets of multi-modal satellite imagery can yield promising predicting performance on multiple time scales (Xu et al., 2026).

2.4. Memory Acceleration

Work in progress

2.5. Research Gap

Attention-based transformers have shown promising results in a variety of disciplines. Especially the diverse learnable embedding vector space serves as an asset due to its seemingly infinite application scopes and ability to learn specific tasks. More closely, the storage of natural phenomena as image-based datasets makes the transformer model an ideal application. Because rasters are pixel data themselves, the bridge from raw data to an embedding that can be used to capture temporal and spatial dependencies between these is especially advantageous. With the ability of transformer models to learn embedding vector spaces in the natural language processing (NLP) domain (as in Figure 2.2), it begs the question whether these architectures are also able to create analogous representations of climate health, a so-called wildfire-risk embedding space.

First applications to wildfire problems show promising results, however only limited work has been done outside of predicting spread. With some research slowly pivoting towards modelling pure wildfire occurrence risk, first bench-marks have proven the theoretical framework possible with valid results.

Furthermore, little to no information is available regarding discussions of temporal scopes of input and prediction scopes. Short-term multi-day predictions of fire spread are an established discipline. However, elongating the temporal prediction scope along with modelling pure risk of occurrence have yet to be investigated. Due to vast amounts of historical data available publically, models can not only learn to track and identify fire hazards, but predict and project their influences into the future.

The main novelty this work presents is the inference of probability of fire occurrence from historical records. By fusing static and dynamic data into a prediction pipeline, the transformer architecture in this work shows the ability to levy spatial and temporal attention mechanisms for predicting wildfire probability. Another gap this research addresses, regards the temporal flexibility of a such predictive mechanism.

This work also investigates both the scope of input data a model uses to form its predictions as well as the temporal scope of prediction and the dependency on model accuracy to make predictions for the distant future (lead-time). Furthermore, an investigation into feature importance via ablation studies is crucial for understanding predictive performance and will represent a second focus of this work.

2.5. Research Gap

As a guide, the objectives defined above have here been defined as guiding research questions:

1. To which extent can spatio-temporal transformers to predict probability of wildfire occurrence?
2. How does the temporal extent of input data influence predictive accuracy?
3. How does model prediction vary under extensive lead-times?
4. To which extent can the importance of input data types be quantified?

Figure 2.3: Research Questions

3. Data

3.1. Data Acquisition

As shown in the introductory sections, transformer models thrive on abundance of data. In order to construct a meaningful model, a large harmonized dataset is necessary. The collection of datapoints used to gather data along with information on temporal and spatial extent and resolution is shown in Table 1:

Endpoint Name	Description	Temporal Res.	Temporal Extent	Spatial Res.	Spatial Extent
MODIS/061/MCD64A1	Global monthly gridded burned area and quality	Monthly	2000-01-01	500 m	Global (90°N to 90°S)
CGIAR/SRTM90_V4	DTM data processed by CGIAR-CSI to fill voids.	Static	2000-01-01	90 m	Global land masses (60°N to 56°S)
ERA5_LAND/HOURLY	High-resolution hourly atmospheric and land-surface re-analysis dataset.	Hourly	1950-01-01	~9 km	Global (90°N to 90°S)
MODIS/061/MOD11A2	Terra MODIS 8-day composite Land Surface Temperature and Emissivity (LST)	8 Days	2000-01-01	1,000 m	Global (90°N to 90°S)
MODIS/061/MOD09GQ	Terra MODIS Daily Level-2G Surface Reflectance (NDVI)	Daily	2000-01-01	250 m	Global (90°N to 90°S)
MODIS/061/MOD09GA	Terra MODIS Daily Level-2G Surface Reflectance (NDWI)	Daily	2000-01-01	500 m	Global (90°N to 90°S)
UCSB-CHG/CHIRPS/DAILY	CHIRPS daily rainfall estimates.	Daily	1981-01-01	~5,500 m	Quasi-global (50°N to 50°S)

Table 1: Overview of data sources

The data used herein stems from a variety of different sensors and data products using varying calibrations and accuracy. Also, since all datasets describe different physical phenomena both the temporal and spatial resolutions must be considered when harmonizing this dataset. The data gathered in this investigation stems entirely from Google Earth-Engine (EE) and was retrieved using API requests.

3.1.1. System configuration

To govern data harmonization, the core defining parameters of the area of interest are globally defined in the system configurations and is passed to each of the downloading and processing steps throughout the process. The interpretation of the area of interest, time,

3.1. Data Acquisition

projection, scale and coordinate reference system (CRS) is handled internally by the *GoldenGrid* class. It inherits start- and end-date from the system configurations and contains a variable list of timestamps for which to retrieve data. Furthermore, the coordinate settings in the inputs are interpreted here as a bounding box structure which earth engine can understand, while also passing along the system CRS and scale.

Add UML
figure of
GG

3.1.2. Earth Engine

The instance of golden-grid is handed to each of the classes responsible for retrieving data from EE. Specifications defined in the golden-grid are directly handed over as query attributes in the request. All cropping operations and projections are done server-side. Resulting rasters are stored for each day in a tagged image file format (TIFF). A script iterates over all endpoints, the product of which is a daily for every module and every day in the time of interest.

3.2. Feature Engineering

In the following steps we take a few important steps from raw data, towards a dataset that a transformer can understand and reliably extract information to learn from. There are a few significant pitfalls when preparing data for a model relying on simple correlation, which will be addressed here.

A choice made to promote easier organisation and fast access of data is to store it in the commonly used Zarr-format. This is a data type can store 3D raster data, making it an ideal fit for time dependent phenomena. Furthermore, it is possible to consolidate data of different modalities into this structure. Zarr data furthermore has the ability to use a method called chunking, which allows for random access and slicing of all three dimensions, here two spatial and one temporal dimension and accross multiple datasets. More specifically, this allows for rapid data selection and loading from disk as opposed to loading georeferenced raster files.

3.2.1. Ground Truth

Most important for a valid prediction mechanism is a valuable ground truth dataset. Chosen for this work is the monthly gridded burned area from MODIS. Despite being delivered on a monthly basis, the dataset contains yearly images which specify the Julian day of a burn event per pixel. The data is processed from a yearly perspective into daily rasters. Essentially, this process creates binary burn maps for every calendar day in the input specifications.

The target extent, a system paramter passed by the config file, defines the temporal extent for which a prediction should be made (e.g. 14 days). We use the previously created zarr containing daily burn records as the basis for this feature. In essence, we use a continuous sliding window that computes the union of 14 consecutive days of burn records and stores it as a new module in the zarr container. These images will later represent the feature which the model predictions are compared to and updated accordingly. As some pixels may never have burnt, we must opt for a maximum value, hence introducing a small bias. Here it was set at 7300 days or roughly 20 years.

3.2.2. Burn History

Another feature which is derived from historical burn records is the burn history map. The idea here is to encode some information into the system, which describes the frequency and temporal behaviour of fire events for every location. As for all other features, we produce a burn history map for every day. The methodology here once again involves reading the binary burn history zarr file and extracting all burn records previous to the desired date. As for one

pixel, multiple fires might have occurred, we compute the amount of days since the last seen event for each pixel. This historical map is computed for all days available in the catalogue and is stored back to a new zarr modality.

By using this kind of information the model has the ability to recognize patterns areas which are highly susceptible to fires. Additionally, we here encode an important idea about burned area, which is that once it has been exposed to a fire, it is highly unlikely to burn again in near time, or might even pose as a natural fire barrier.

Figure of
zarr slicing
and union

3.2.3. Road Proximity

As discussed in Section 2, the main driver of wildfire ignition is human activity. This information is implicitly encoded via proximity to roads. The basis of this serves a vector dataset that encodes roads for the entirety of Europe, which was synthesized as part of a study to describe patterns in road infrastructure (Meijer et al., 2018).

The process here includes selecting roads in the area of interest and creating a small buffer around them, essentially converting from *LineString* to *Polygon*. These are then converted in a binary road map raster, which has exact dimensions as all other inputs, as dictated by the golden-grid. Ones indicate roads, zeros indicate any other land use. On the basis of this we conduct a grid search for every pixel in the image. Here we compute the physical proximity from the center of each pixel to the nearest road pixel.

Figure of 3
step process
here?

3.2.4. Ratio-Scale data types

Processing the digital terrain model (DTM) as a raw feature is possible, and is valid, however we enrich terrain information passed to the model by additionally including derivatives of it. After all it is less the total altitude a model is interested in when predicting fire, but more the contextual appearance. Therefore, the terrain is processed into slope and aspect maps.

Slope is simple to compute as for every pixel we simply conduct a grid search of the immediate pixel neighbors and compute the joint maximum slope for each position. This feature is crucial to fire modeling, as fire propagates well in highly slanted terrain. Adding a representation of aspect, which is valuable to the model, is more complicated. Aspect is denoted as an angle, which is an interval scale as per Equation (3.1):

$$\varphi \in [0, 360) \quad (3.1)$$

A transformer whose mechanisms rely on correlation and scaled dot-products is inherently flawed in determining correlations of such interval scales. As an example an aspect of 355° is quite similar to 0°, however given on a ratio scale, these are far apart, these would yield a low correlation. To circumvent this effect, the aspect dataset is split up into two components, one describing the east-west component and north-south for the other. Both run on a ratio scale, together describing the angle of aspect (Figure 3.1). The two aspect and slope datasets complete the suite of terrain information. It is up to the transformer to find spatial correlation between these in order to gain an understanding.

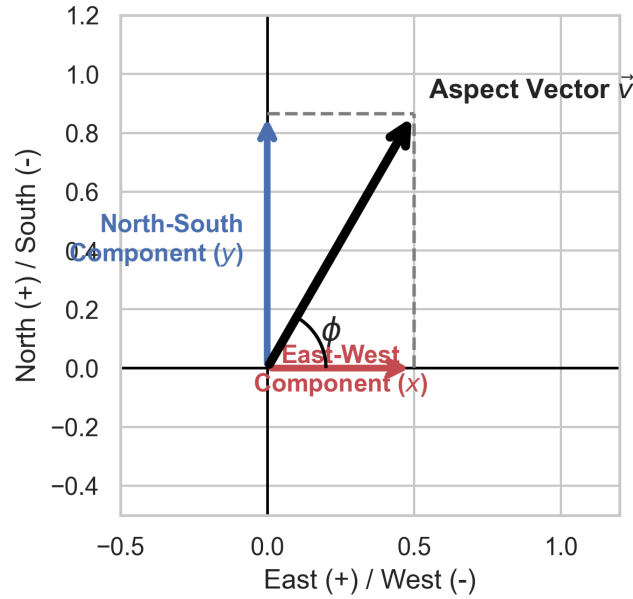


Figure 3.1: Decomposition of Ratio scale datasets into components

The same process is carried out for the wind datasets. Daily data is delivered via a cyclical wind-direction, and wind speed map, which both together denote the wind vector. As for the terrain we dissect wind direction into east-west and north-south component and deliver them to the transformer as separate datasets and allow it to cross-correlate the information back together.

3.3. Processing

With all information processed into value scales a transformer can interpret, we must additionally normalize datasets. The choice here was made to normalize all values to the $[0, 1]$ domain. This was done by taking the entirety of 10 years of data available for each data module and recording the global maximum and minimum values for each. Next, every image in the respective module was normalized by this metric. Ruthlessly merging metric scales into a normalized domain might seem counter-intuitive at first, but is justifiable and follows very concrete reasoning. Primarily, the SI-derived units in which the datasets are measured in are mostly arbitrary. The transformer model cannot distinguish the extent to which precipitation, measured in mm, is relevant to land surface temperature ($^{\circ}\text{C}$). Furthermore, combining datasets which include values in different orders of magnitudes will lead to processing instability in the transformer. When cross-attending elevation, which in this carries varies between $[0, 2500]$, and land surface temperature, the weights that correlate these values to one another will have to scale up and down drastically to achieve stable cross attention. This also shows, that were values not to be normalized, the transformer de facto will do it nevertheless.

We additionally must treat the datasets missing data points. Many of the datasets are influenced by cloudcover. Those relying on near-infrared data like NDVI, which does not penetrate cloud cover long stretches of missing data can be observed especially in the winter months. Regardless of its relevance to a wildfire prediction, we additionally mask out waterbodies on all images. Not all modularities produce values over oceans and rivers, making it a safer option to remove them entirely. A joint no-data value both for missing data and water-cover of -1 has been chosen as a unified value. Through its learning process, the transformer will learn to give especially those pixels over water less weight, effectively removing them from the data scope.

Figure 3.2 shows the final resulting input dataset for one day as it would appear to the transformer.

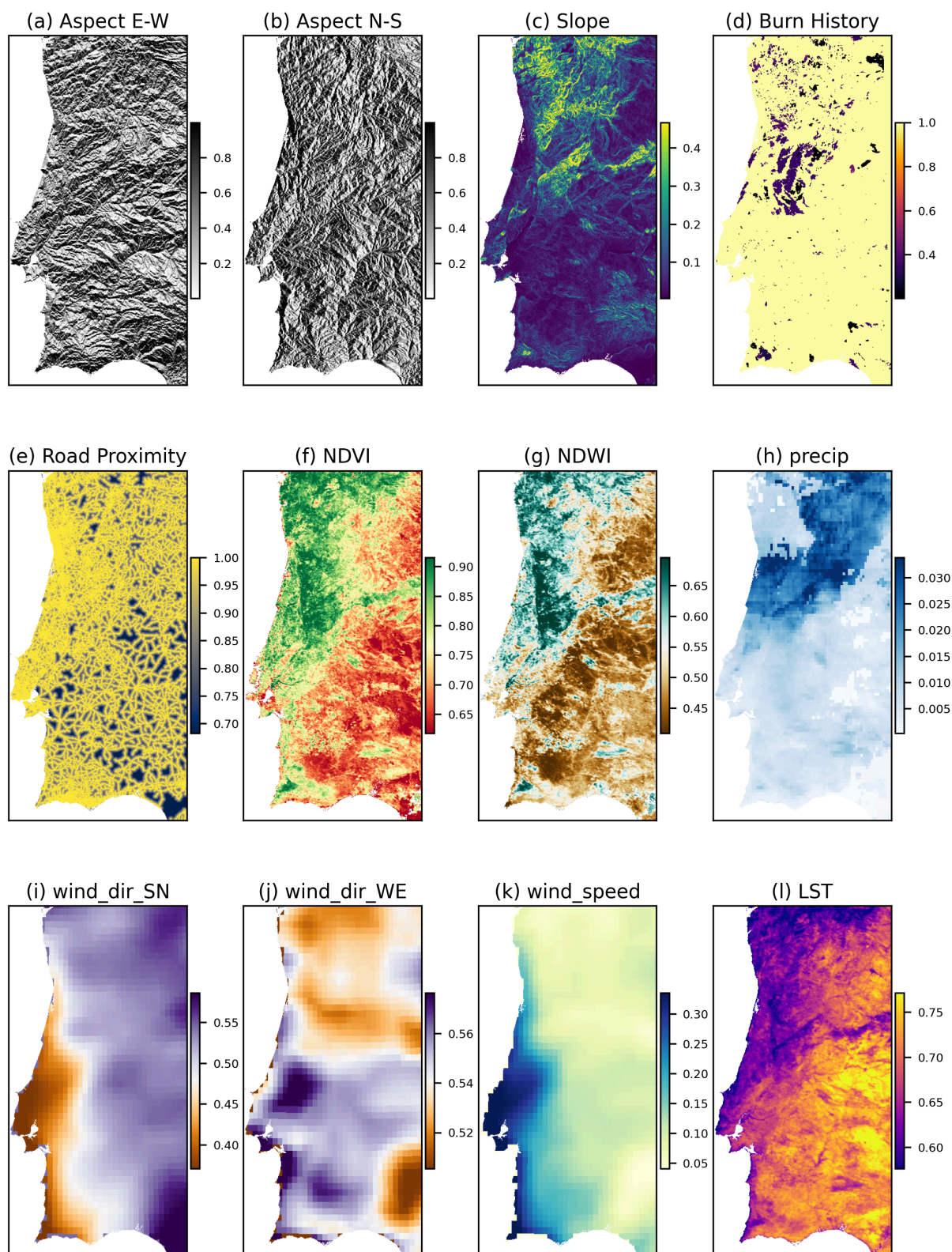


Figure 3.2: Snapshot of input data types

3.3. *Processing*

4. Methodology

This section provides an overview of the methodological choices made in the investigation. First, a short account of the system overview is given, before an account for the construction of data into datasets is made. Further, a complete walk-through of the model architecture is given. Finally, the integration of the model into a training workflow is explained.

4.1. Workflow & System Overview

work in progress

```
data:
  spatial_extent:
    longmin: -9.4900
    longmax: -6.0234
    latmin: 36.812
    latmax: 42.2724
  crs: "EPSG:3763"
  scale: 1000 # meters
  temporal_extent:
    start_date: "2020-01-01"
    end_date: "2024-12-31"
    day_interval: 1
  sequence_period: 4 # Time between respective sequences
  sample_extent: 10 # Total time of sample
  sample_period: 1 # Time between images in sample
  target_extent: 4
  lead_time: 0 # Time between last sample and target
```

4.2. Dataset Construction

4.2.1. Timeframe Configuration

A section of the research objective is to assess the influences of the temporal scopes of both data and predictions. Both can be variably adjusted depending on the desired test case. As we handle temporally dynamic datasets we need to define how timesteps of samples are represented in the dataset. To ensure consistency in naming these, we here declare a series of term definitions relevant for this step:

- **Sample:** Input data the model sees to form predictions; describes the set of timestamps for one forward pass
- **Target:** The collection of images, that is variable length, that is used for testing the model predictions
- **Sequence:** A pair of a sample and target
- **Dataset:** A collection of sequences
- **Sequence Period:** Separation between individual unique sequences
- **Extent:** Used both sample and target, defines their respective variable length
- **Lead Time:** The time between the last seen timestamp in the sample and first one in the target. In other words: the amount of time the model predicts into the future

Update figure of timestamp configurations

Every sample is constructed by 3 different kinds of data, that are read, packaged together and channeled into the model:

- **Static:** equivalent in every sample, describes non-variable data (e.g. terrain)

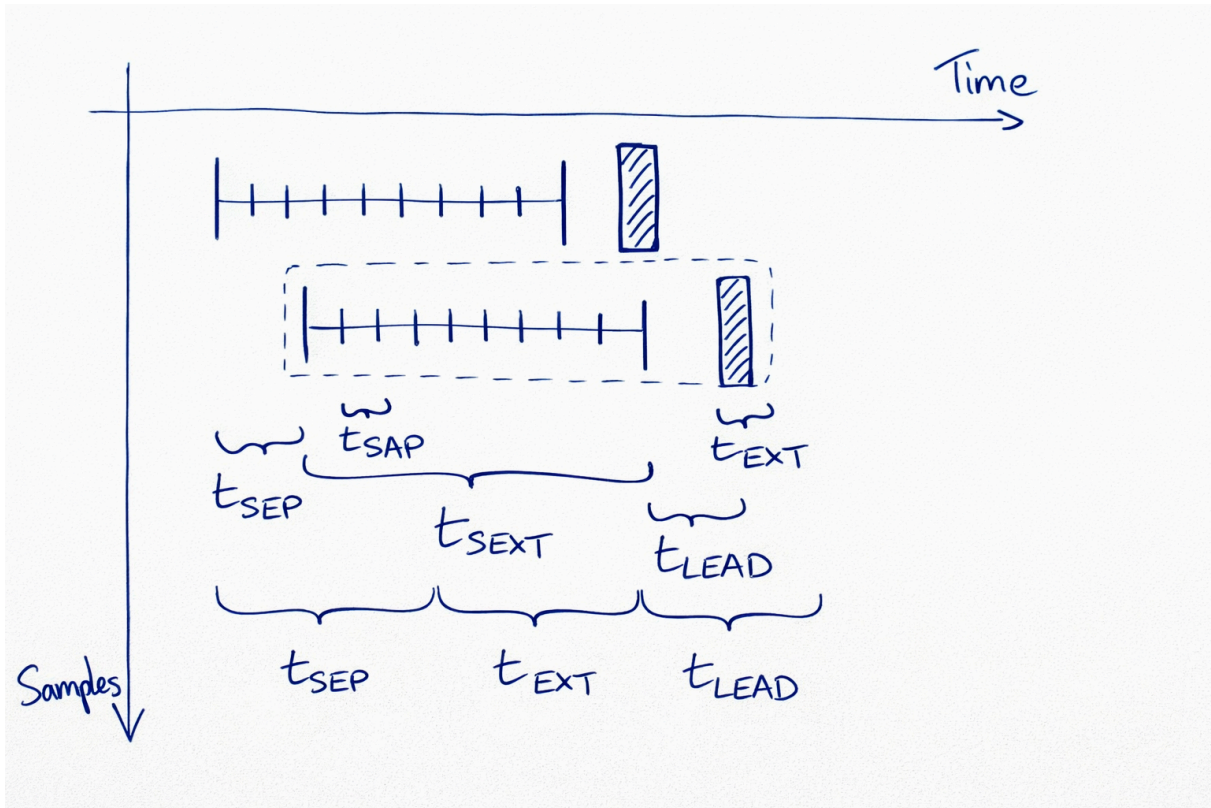


Figure 4.1: Definitions of temporal scope in project datasets

- Dynamic: time-series data consisting of multiple images (e.g. wind)
- Single-dynamic: dynamic data that is less temporally dependent and assumed static over the extent of the sample (e.g. 30-day precipitation sum)

Figure 4.1 illustrates the terms defined above. It shows two consecutive sequences as well as their respective samples and targets. It shows the temporal division of the sample into subsequent timesteps. The distance between sample and target corresponds to the above definition of *lead-time*.

The configuration file that belongs to a model training governs the total time of interest and all parameters defining the temporal relationship of input data. The dataset builder then generates a set of timestamps as governed by the rest of the temporal configurations and iterates through the respective file for every module. Sequences with missing images are skipped entirely to ensure consistency. These dynamic modules are accompanied by the static data, which is the same in every sample. The result is a set of sample/target combinations spanning the entire domain of temporal interest.

4.2.2. Torch Datasets & Tensors

Moving data from images into the Pytorch framework is done using its fundamental data structure: the **Tensor**. Like in the traditional mathematical sense this is an array of arbitrary dimension containing elements of a single data type but has a few computation-driven features. Most importantly, it is optimized to be handled by GPU. Tensors are directly associated with the model instance (elaborated in Section 4.4.2) and have an attribute that defines whether they are adjustable model parameters. This concept is specifically designed for those tensors containing model weights. PyTorch can track the operations imposed on a tensor, which makes later backpropagation to these easier (Pytorch-Contributors, 2019a). The sequences described

in Section 4.2.1 are appended into such a tensor structures and then saved as portable datasets to be made accessible on larger compute resources.

4.3. Model Architecture

In order to motivate the structure of the here applied transformer, this section walks through the sequential steps incoming data takes through the model. This represents one forward propagation. The later showcased training regiment makes this forward pass once for every batch in the training data, thus cycling through it an abundance of times.

4.3.1. Spatial Encoding

The initial objective of the transformer model revolves around turning input data into tokens by segmenting it into equal patches (e.g. 16x16 pixels). The patch size, a model hyperparameter, defines the pixel extent of square patches every input image is divided into. As we handle only single-band data we treat the image as a grid of height and width: $h \times w$.

This is realized via a 2D convolution with stride equal to the desired patch size. This ensures zero overlap between neighbouring patches. An important element to consider is the fact that both height and width of images need not be divisible without rest by the patch size. To counter this we use reflective padding, a process which mirrors input images around their edges to fill in pixel data until the next complete patch. Extending height and width to h_{pad} & w_{pad} allows for perfect image division into patches. As the model input consists of multiple modules M for consecutive time steps t that are processed in batches B of variable sizes the input tensor has dimensionality:

$$\tilde{T} \in \mathbb{R}^{B \times M \times t \times h_{pad} \times w_{pad}} \quad (4.1)$$

Every image of dimension $h_{pad} \times w_{pad}$ is then subject to the flattening operation, here illustrated with a patch size of 16 pixels.

$$P \in \mathbb{R}^{16 \times 16} \rightarrow P_{flat} \in \mathbb{R}^{256} \quad (4.2)$$

Up to this point, pixel values have not changed, simply the indexing is altered. To produce the embedding, the convolution operation then maps every flattened patch into the embedding dimension. Here we showcase the operation for a patch size of 16 pixels, mapping to the embedding of dimension 128:

$$e = wP_{flat} + b \quad (4.3)$$

where

$$w = \begin{pmatrix} w_{1,1} & w_{1,2} & \dots & w_{d_{patch},1} \\ w_{2,1} & w_{2,2} & \dots & w_{d_{patch},2} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d_{embed},1} & w_{d_{embed},2} & \dots & w_{d_{patch},d_{embed}} \end{pmatrix} \in \mathbb{R}^{128 \times 256} \quad (4.4)$$

The convolution weights are learnable and are only shared between samples of the same module. Every module is assigned an independent convolution operation and individual weights.

4.3. Model Architecture

4.3.2. Temporal encoding

The model architecture demands a separate routing of static and dynamic data channels. All dynamic modules with multiple time steps receive a temporal encoding that is an extension to the one described in Section 4.3.1. The objective of this is to give patches that describe the same physical area in different time steps a closer connection to one another. This is done by treating the different time steps for one module together. The single-image 2D convolution is replaced by a 3D convolution that treats the series of images of timesteps t as a 3-dimensional set of values and projects it into the embedding dimension d_{embed} :

$$\mathbb{R}^{t \times d_{patch} \times d_{patch}} \rightarrow \mathbb{R}^{d_{embed}} \quad (4.5)$$

The convolutional operation is analogous to Equation (4.3) for a static embedding with a kernel of dimension:

$$w \in \mathbb{R}^{d_{embed} \times 1 \times P^2} \quad (4.6)$$

In essence, this step results in a token for every patch in every timestep of every image. This effectively patches every timestep in every module independently. The objective here is to prevent the immediate blending of time steps, as this is the objective of the transformer layers at a later stage.

4.3.3. Spatial Embedding

Having obtained a set of embeddings for all images of varying modalities in the model input, this part of the transformer architecture intends to introduce spatial and temporal relationships between the patches called *embeddings*. These greatly influence the magnitude with which patches learn from each other.

The spatial embedding is applied to every patch regardless of being temporally dependent or not. The idea is to assert a learnable idea of proximity from one patch to another. As wildfires are localized phenomena, the the objective of the spatial embedding here is to give neighbouring patches a strong correlation. Distant patches, while sharing lower correlation, can still influence one another, with limited effect. The important difference to a convolution here is that we do not simply rule out distant relationships but keep them contained. Rather than imposing a relationship on the transformer, we give it rough guidelines and freedom to give high weight to the relationship even between distant patches (Heo et al., 2024). Having created a notion of spatial similarity is crucial for the later attention process in which more similar patches will conduct higher weighted attention with one another.

In order to create such a relationship we come up with an individual positional encoding for every patch that is added onto each embedding:

$$\vec{e}_{token}' = \vec{e}_{token} + \vec{e}_{pos} \in \mathbb{R}^{d_{embed}} \quad (4.7)$$

To create a positional embedding that retains similarity between proximal patches, we use sinusoidal positional embedding. By imposing sine and cosine waves of varying frequencies over the patch grid, we are able to establish a unique encoding for every patch that has distinct relationships to other patches, depending on their Euclidean distance. This is done by first defining a set of frequencies with $d = \frac{d_{embed}}{4}$:

$$\omega_i = \frac{1}{10000^{\frac{i}{d}}} \quad (4.8)$$

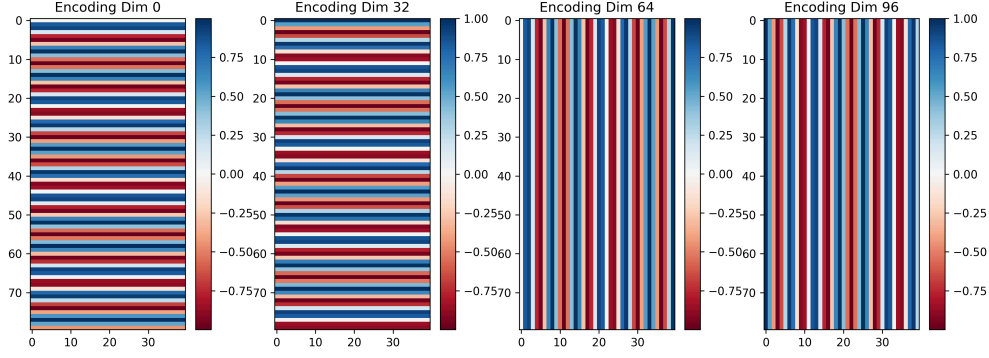


Figure 4.2: Additive encoded values per encoding dimension

The nature of the integer 4 is that in order to create the most unique positional embeddings, we split the added partial embeddings into both height and width and also sine and cosine categories. We scale the frequencies with 10000 in order to fit multiple periods into the image. Interpreting height and width as coordinates we can thus describe the positional encoding of a patch as the vertical concatenation of these frequencies:

$$\vec{e}_{\text{pos}} = \begin{pmatrix} \sin(h\omega) \\ \cos(h\omega) \\ \sin(w\omega) \\ \cos(w\omega) \end{pmatrix} \in \mathbb{R}^{d_{\text{embed}}} \quad (4.9)$$

Figure 4.2 shows samples of the modes of the positional embeddings for every patch in the input image, in this case 80×40 . These include waves that propagate in direction of height and width. For every patch we add the embedding values of all dimensions.

Given the multitude of modes, we can form very individualised encodings for all patches. Figure 4.3 shows the cosine similarity of patches to one another for different embedding dimensions as demanded from the spatial embedding. Every pixel in the images represents a single patch. Here, the central patch serves as an anchor-patch, with the values of all patches describing its similarity to it. This shows that sinusoidal positional embeddings are able to establish similarities and differences between patches, depending on their respective proximity. Furthermore, it is important to mention that the utility of this process heavily depends on the embedding dimension chosen. A higher embedding dimension allows to pass more sinusoidal modes and hence to create a more distinct characterization of each patch. We here show the cosine similarity for three embedding dimensions. We detect a clear trend that when treating choosing higher embedding dimensions will lead to sharper definitions of similarity and differences of patches.

4.3.4. Temporal Embedding

Weighing patches of the same modality against each other should be entirely up to the transformer model. Afterall, the idea is to find nuanced temporal relationships both within a modularity and between pairs of them. To enforce distinct temporal relationships between dynamic tokens as well as a global perception of the model for seasonality we include an additional temporal encoding for all dynamic modules. Used here is an encoding of the Julian day, which is simply an integer number of the day of the year (0-365). To make use of this in the best way, we encode the Julian day so that it captures the true nature of the climatological

4.3. Model Architecture

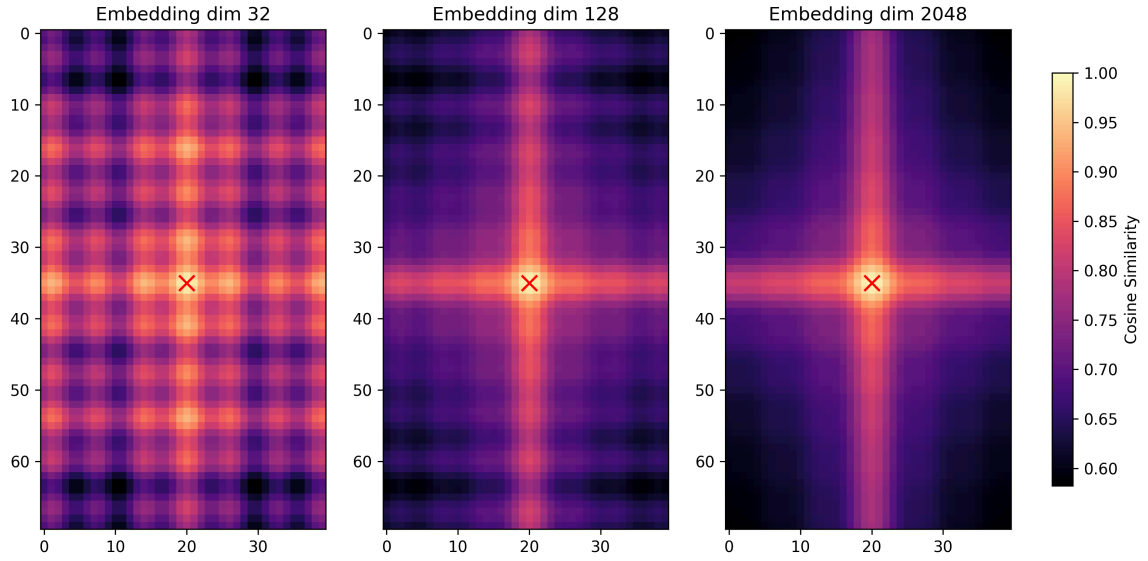


Figure 4.3: Cosine similarity of patches relative to central anchor-patch at varying embedding dimension

conditions: its cyclical nature. The embedding of a patch from a day in December should be close to one in the beginning of January, despite their Julian day being far apart.

Making such an embedding is realized through a two-step process, first mapping the Julian day J_t into a clock-like cyclical two-dimensional space and then translating it into the embedding dimension. The translation into a cyclical representation v_t is done using a sinusoidal projection via:

$$v_t = \left[\sin\left(\frac{2\pi J_t}{365.25}\right), \cos\left(\frac{2\pi J_t}{365.25}\right) \right] \quad (4.10)$$

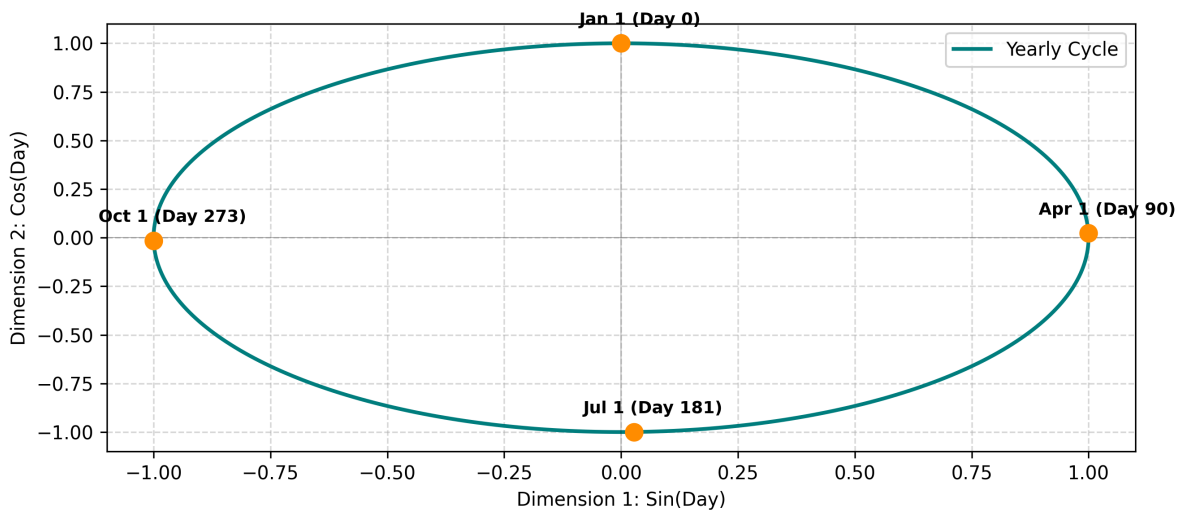


Figure 4.4: Julian Day encoding into 2D vector space

Figure 4.4 shows the representation of each Julian day in a two-dimensional vector space, effectively capturing cyclical relationships of months even with distant Julian days. The next step entails moving this to the same embedding dimension which the tokens occupy. Simi-

larly to convolution processes shown earlier, we apply a learnable weight matrix to this vector representation projecting these values into $\mathbb{R}^{d_{\text{embed}}}$:

$$\vec{e}_t = W_{2 \times d_{\text{embed}}} \cdot \vec{v}_t + \vec{b} \quad (4.11)$$

Having embedded all tokens of different modalities into embedding space, we additionally give each token a modality tag $\vec{s}_m$. A tag is unique per module and is a learnable parameter with the objective to give the attention heads a clearer idea of where tokens came from. Throughout the learning process the transformer will be able to differentiate between different modules as the tag injects a numeric meaning to all tokens.

The final transformer-ready embeddings for static and dynamic tokens are additions of their individual spatial, temporal and static embeddings like so:

- Static Embedding: $\vec{z}_n^m = \vec{e}_{\text{token}, n}^m + \vec{e}_{\text{pos}} + \vec{s}_m$
- Temporal Embedding: $\vec{z}_{n,t}^m = \vec{e}_{\text{token}, n}^m + \vec{e}_{\text{pos}} + \vec{e}_t + \vec{s}_m$

Simply adding large vectors to our representations of patches might seem counterintuitive at first, however it is crucial to remember that the representations of data modularities are learnable. The longer the transformer architecture trains, the more it knows to contextualize a patch of precipitation data differently, depending on the time of year. The most elegant analogy to this process is music. If the raw encoding of a patch is analogous to a melody, the positional and temporal encoding represent the harmony accompanying it. Music is propagated as the sums of interfering frequencies, however they are discretely distinguishable to the human ear. The same is true for the transformer, it learns to contextualize the raw encoding (melody) given an extra set of information, the embeddings (harmony).

4.3.5. Token Unification

The heart of the concept in this model relies on early fusion. Rather than separating tokens per module, this approach allows for patches accross time and modularity to freely communicate with all others. Despite having the drawback of computational load here, this is a crucial design choice with the purpose of not overconstraining the system. With respect to wildfire modelling, this is a great asset. Data modularities will have varying levels of temporal dependencies. As an example, while the last timestep of wind speed might be sufficient to cast a valuable prediction, the land surface temperature might need to be tracked over the course of a few days to cause wildfire favoring conditions. The setup of keeping all tokens of all modules separate rather than concatenating them early allows for exactly these dependencies to be found in attention.

The token unification step therefore concatenates embeddings into a joint sequence, as a preparation for the next step. Crucially, the order of tokens is of no importance here as the attention steps that follow are invariant to permutation of tokens. Furthermore, due to the spatial and temporal embeddings and embedding tags, tokens are uniquely identifiable. The tensor at this stage holds embeddings accross all timesteps, patches and modules in one place:

$$\tilde{T} \in \mathbb{R}^{B \times [h \cdot w (n_{\text{dyn}} \cdot t + n_{\text{static}})] \times d_{\text{embed}}} = \mathbb{R}^{B \times n_{\text{token}} \times d_{\text{embed}}} \quad (4.12)$$

4.3.6. Joint Attention

The tokens in the concatenated tensor at this stage query each others information for the first time. This step can be considered self-attention by mathematical standards, as all embeddings reside in the same tensor, however this really allows embeddings to query one another accross modules. This joint encoder is composed of 8 identical blocks, consisting of 8 attention heads

each, equating to a total of 64 attention heads. Each of these have the ability to learn individual feature relationships.

To motivate the design choice of the join attention block here, a short revisit to the core idea of the original objective of cross-attention for translation learning tasks. A sentence, in which every word forms a token, will likely be rearranged in the translation to another language (see Figure 4.5). In the English language we expect the embedding of *red* to have a large influence that of *fire* and likewise in its French counterpart. When cross-attending these tokens of the two datasets, we expect no consistency in the arrangements in the positions of directly translatable words. Therefore, a cross-attention mechanism considering all tokens accross space and time is mandatory.

EN: The red fire spreads across the land
FR: Le feu rouge se propage à travers le pays

Figure 4.5: Example of token order of two cross-attending datasets

The same concept is true for a spatio-temporal transformer. Rather than separating modularities and conduction isolated self-attention, we consider all possibilities. An example here could be the effect of precipitation on future burn probabilities. When predicting for wildfires, the further the latest rain surge was recorded, the higher the probability for an event will be. This underlines that allowing tokens to attend accross time and modules is crucial at this point.

An indispensable mechanism that is implemented at this position during training is dropout. When learning, models tend to follow their most probable direction of learning, meaning once it locked onto characterizing a specific learnt feature, this is all it will be able to understand. Dropout, systematically works against this, as the goal is not to understand just single relationships between data, but as many as possible. Therefore, at random intervals and attention heads, dropout will temporarily zero-out all attention weights. This signifies to the encoder that for this specific iteration of epoch it will not have access to relating pieces of data in the exact way it is used to. Consequently, it will have to rely on making new connections.

4.3.7. Disaggregation

Having attended the modular tokens, the goal is to reshape the obtained information into a prediction map. Important to consider however is that despite having obtained a set of learned tokens, they still stem from different modalities. In order to make use of dynamic and static data in the best way, the tensor contained all attended global embeddings, is disaggregated back into dynamic and static tensors. The patches from dynamic modules are first averaged accross the temporal dimension:

$$\mathbb{R}^{B \times M \times t \times d_{\text{embed}}} \rightarrow \mathbb{R}^{B \times M \times d_{\text{embed}}} \quad (4.13)$$

It might seem that information is lost via this operation. However, when considering the previous attention block, which has the purpose to weight patches accross timesteps by their importance, this step only aggregates this information into one time step. During training, patches from the more relevant timesteps will receive higher weights, thus the averaged representation of the time series will represent this.

Tokens of static and dynamic layers are consequently concatenated into tensor T_{decoder} , setting up for the decoder:

$$T \in \mathbb{R}^{B \times n_{\text{token}} \times d_{\text{embed}}} \quad (4.14)$$

4.3.8. Decoder Layer

The final task of the vision transformer is to take the embedded vector representations in the embedding space and project them back into a form which represents our ground truth: a 2D image.

It is of course possible to make predictions for every patch. This is however not just an oversimplification but also does not provide the opportunity to predict for as many samples as possible. Table 2 shows the complete structure of the decoder process that projects back to image space:

Step	Tensor Size $\bar{T}$	Description
Input	$B \times n_{\text{token}} \times d_{\text{embed}}$	Output of attention blocks
Reshape 1	$B \times d_{\text{embed}} \times h_{\text{pad}} \times w_{\text{pad}}$	Reshape back to 4D feature map
Convolution 1 + Batch-Norm / ReLU	$B \times \frac{d_{\text{embed}}}{2} \times h_{\text{pad}} \times w_{\text{pad}}$	3×3 kernel with padding, reduce channel depth
Upsample 1	$B \times \frac{d_{\text{embed}}}{2} \times 4h_{\text{pad}} \times 4w_{\text{pad}}$	$4 \times$ scale, bilinear interpolation
Convolution 2 + Batch-Norm / RELU	$B \times \frac{d_{\text{embed}}}{4} \times 4h_{\text{pad}} \times 4w_{\text{pad}}$	3×3 kernel with padding, reduce channel depth
Upsample 2	$B \times \frac{d_{\text{embed}}}{4} \times 16h_{\text{pad}} \times 16w_{\text{pad}}$	$4 \times$ scale, bilinear interpolation
Convolution 3	$B \times 1 \times 16h_{\text{pad}} \times 16w_{\text{pad}}$	1×1 kernel, pixel-wise classifier
Slice	$B \times 1 \times h \times w$	Un-pad operation to original input size

Table 2: Stepwise decoder mechanism & prediction head

In this stage we use a progressive resolution decoding. After first reshaping the tensor into four dimensions, equal to the dimension of the output we require, a series of convolution and upsample steps are used. Each convolution halves the dimension of the embedding, after which the upsample redistributes this via bilinear interpolation to widen the sizes of the patches back into a full-size image. Finally, the decoder features the *un-pad* operation which discards the padding created initially to prevent data disturbance.

4.3.9. Model Parameters

By design, the total amount of parameters in the model is dependent on the temporal extent of the sample, as every patch and every timestep receives an individual encoding. The model uses 90% of its parameters on encoding while the remainder is equally distributed on embedding and decoding. Table 3 shows the total parameters for a baseline model with a sequence extent of 5 days.

Module Name	Parameter Count
static_embeds	49,280
dynamic_embeds	57,472
join_backbone	1,586,176
sequence_projector	16,512
decoder	92,481
Standalone Parameters	1,536
TOTAL	1,803,457

Table 3: Model Parameters per class

4.4. Training Routine

The setup of the training regimen of models is crucial as without certain guidelines, the model could make faulty predictions. The most important vocabulary in this regard is *overfitting* and *generalization*. Generalization describes the ability of the model to transfer its learned patterns from training data to previously unseen data. This showcases the ability to concretely learn the physical underlying the data, rather than simply memorizing it. A model is said to be overfitting, if the prediction results rely on the training data including all its inaccuracies and noise rather than actual representation of data. The training routine used herein caters to both of these issues. As a guide, Algorithm 1 shows the pseudo-code implementation of the training regiment.

To promote generalization of a trained model, we use a train-validation-test split of 70-15-15 respectively. These are the proportions the dataset is split into at the beginning of training. Every dataset has a distinct purpose. The training data is seen by the model on every iteration, losses are computed accordingly. The model achieves a more refined understanding of this training data in every epoch, however there is a minute distinction to be made here. We must differentiate those model improvements in the loss to those which describe a model gaining an understanding of physical phenomena and those in which it is taking a short-cut and is learning the actual data training data. The safeguard for this is the validation data. This data has no effect on the model. The losses of the model are computed against it after every epoch. We therefore only consider a model to be improving after a new lowest training loss, if the validation loss has also improved. A model cannot learn the validation data, hence if its validation loss is decreasing, it must be learning the science and not the data. The testing data is only used once at the end of the training run. Its entire purpose is to ensure the global generalization of the model. If it is vastly greater than the best validation score, the model is over-fitting to its input data.

```

1 Input: Load dataset, initialize dataloader, move to GPU
2 Calculate class imbalance positive weight  $w$ 
3 Initialize loss function, optimizer, model from config
4 Split data (70/15/15) into training- validation- and testing sets
5 best_loss =  $\infty$ 
6 for epoch in num_epochs:
7     train_loss = 0
8     for batch in train_data:
9         Predict for batches given static & dynamic inputs
10        Compute losses based on WBCE, predictions & ground truth
11        Set optimizer gradients to zero
12        Backward pass
13        Update weights
14        train_loss += batch_loss
15    Compute: validation loss on validation dataset
16    if validation_loss < best loss:
17        best_loss = validation_loss
18        Save model
19    if no improvement for 7 epochs:
20        break
21    Compute: test loss on test dataset

```

Algorithm 1: Training routine

4.4.1. Loss Function

Following a successful forward pass, we evaluate the accuracy of the model by comparing its prediction to the ground truth as defined by a loss function. The main specification the loss function has to adhere to in this problem set-up is the type of target domain in its binary form (fire / no fire). The other is the lopsidedness of our target categories. Due to the very rare nature of fire events, most of the target data describes no-fire, or to the model: zeros. We therefore choose a categorical loss function that in addition to being sensitive to class imbalances is scaleable by a custom weight, the weighted Binary Cross-Entropy loss function (WBCE). It depends on the model prediction p_i , the ground truth y_i and positive weight w . The loss function sums N comparisons in total, one for every pixel in the ground truth.

$$L = -\frac{1}{N} \sum_{i=1}^N w y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \quad (4.15)$$

This setup allows for customizable weighing of output cases. As a concrete example, Equation (4.15) allows us to treat categorical errors made by the model differently. The penalty given by the loss function should be much higher for an incorrect prediction of high-likelihood of fire than a missed prediction of a fire as the no-fire category dominates in overall frequency. This also prevents the model from simply predicting zero to all images, in which case it would still be correct in most cases, however wildly missing the point of its training exercise.

Figure 4.6 shows the loss plane for all possible prediction/ground-truth combinations and a class imbalance $w = 5$. This illustrates a zero loss for all correct predictions along with the two

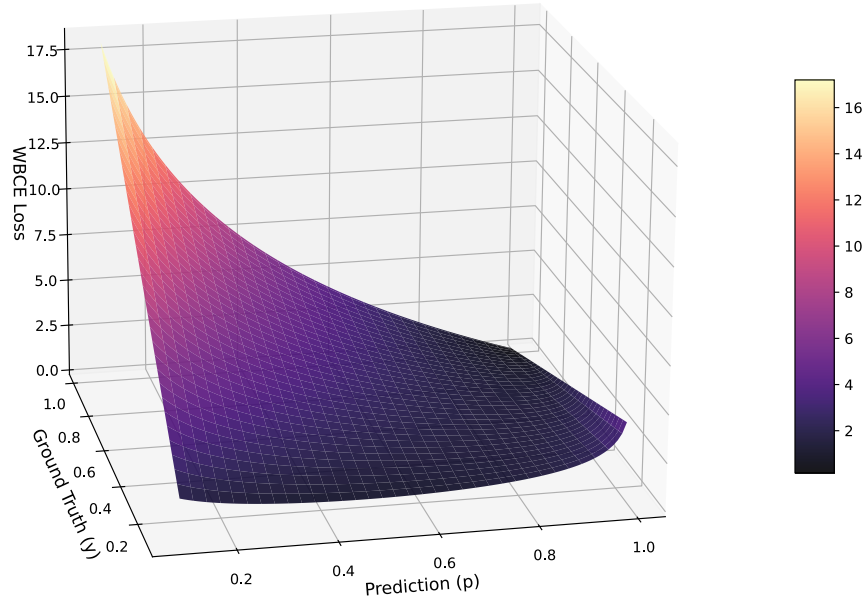


Figure 4.6: Loss surface of Weighted Binary Cross Entropy (WBCE) loss function

extreme parts of the domains in which the model has made an error. By predicting less than the actual ground truth the model is consistently penalized harsher than vice-versa.

Parameter choice of the positive weight w is determined by the entirety of the training dataset before training and is then passed as an input parameter. In essence, all ground truth images are summed to yield a balance of categories.

4.4.2. Back Propagation & Gradient Descent

The goal of the training regimen is to produce the combination of model parameters that results in the least amount of loss, more specifically a minimal validation loss. To achieve this, the loss is evaluated against all model parameters after every batch. This optimization problem can be thought of a local or global minimum in the *loss-landscape* of $L(w_1, w_2 \dots w_i)$ to a model parameter combination that produces a lower loss, the following steps work to descend along the steepest gradient of this landscape. To do this, we calculate the gradient of the loss due to each model parameter:

$$g_t = \frac{\partial L}{\partial w_i} \quad (4.16)$$

This tries to approximate the extent to which the loss, which describes the discrepancy between prediction and target, is due to this specific parameter. Parameters with a high gradient shall

be updated more aggressively than those being responsible for little loss. The method for updating these model parameters is accompanied by a further parameter, the learning rate η . This dimensionless value scales the overall aggressiveness of model updates. It's value heavily influences the stability of the training process. If chosen inadequately, the model may never find an equilibrium state. The updated weight parameters θ are sums of their original values and the loss gradient, scaled by the learning rate:

$$\theta_{t+1} = \theta_t + \eta g_t \quad (4.17)$$

4.4.3. Optimizer & Learning Rate Scheduler

Pytorch uses to further implementation to aid this model weight update process. First is the learning rate scheduler, in this case we use the OneCycle learning rate scheduler. This system is designed to dynamically change the learning rate to a dynamic one that changes with every epoch (η_t). The job of this specific scheduler is to start with a lower learning rate and grant the system so-called warm-up epochs. This caters to one specific problem that is common in machine learning model training and specifically one of this kind which deals with high class imbalance. The model weights, as initialized before the first training loop are random and represent no coherence with the data or reality. Applying a full-scale learning rate to these can lead to erratic loss jumps in subsequent epochs. Using the earlier landscape analogy this corresponds to overshooting a downhill section in a landscape. Once the training process has found its momentum, the scheduler will gradually increase the learning rate to a higher value. This is the stage in which the gradient descent process has gained its momentum, and we require computed loss gradients to have maximum effects on the model parameters. It is this exact stage in which the model learns most about its tokens. As the loss function approaches its local minimum the scheduler ideally decreases the learn rate again, as not to overshoot the minimum. The effect of this is a user setting and is proportional to the total amount of epochs the system is in training for.

Another crucial implementation aiding training performance is the optimizer. Here chosen was the *AdamW* optimizer. This creates a valuable synergy with the learning rate scheduler to produce and enhanced image of how weights should be updated, following their loss computation. Along with the gradient a weight produces for the loss, the optimizer tracks momentum vectors $\hat{m}_i$ for each weight over the course of a few epochs. The logic insinuates that a parameter improving in the same trajectory for subsequent epochs, will likely continue in the same direction. It can therefore confidently receive a larger update as compared to a parameter that has recently stagnated or fluctuated. Together with the scheduled learning rate the weight updates postulated in Equation (4.17) becomes:

$$\theta_{i+1} = \theta_i - \eta_t \cdot \left(\frac{\hat{m}}{\sqrt{\hat{v}_t + \varepsilon}} + \lambda \theta_i \right) \quad (4.18)$$

Another parameter $\hat{v}_i$ tracks the erraticism a parameter shows during epochs. Those values behaving unpredictably will receive suppressed updates while those showing consistency will be rewarded greater updates (Kingma, 2014; Pytorch-Contributors, 2019b).

4.5. Experiments

The aim of this study is mainly to create such a spatio-temporal transformer model that serves as a predictor for wildfire events. More specifically, focus will be given to investigating the temporal dependencies of both the input sample and output target. Investigation into how

4.5. Experiments

these drive accuracy will be given. Furthermore, it is crucial to understand feature importance. Therefore, the ladder part will focus on extracting feature importance.

To guide the reader through the results of the research, every research question, as defined in Figure 2.3, will be addressed concretely by an experiment in Section 5:

- **RQ 1** → Experiment 1: Baseline Model
- **RQ 2** → Experiment 2: Varying sample/ target length
- **RQ 3** → Experiment 3: Lead time horizon stress test
- **RQ 4** → Experiment 4: Ablation studies

5. Results

To guide through the results of this investigation, the research questions formulated in Figure 2.3 will continuously be referenced in the following section to ensure concrete discussion of the defined goals.

5.1. Experiment 1: The Baseline Model

As a proof of concept and a discussion of **RQ 1** in Figure 2.3, here follows a description of the choices made to yield a working model that can confidently predict wildfire occurrence probability.

All model training was conducted according to the specifications in Algorithm 1. The objective of it is to obtain a set of model parameters, that produce a highly skilled, but generalized prediction mechanism. With the subdivision of data into training, validation and testing data a few important guidelines were implemented here to ensure valid training performance of the models. First, the training loss should be a smoothly converging curve, asymptotically approaching a minimum. The same goes for the validation performance for each epoch. Once validation loss no longer improves but starts to fluctuate or increase, the model has shifted from learning underlying process to learning the training data. This can be seen in most training performance curves and is the condition under which training is stopped. These will later be presented in detail for each sub-experiment for this investigation.

The most troublesome phase of training in general was the initiation phase of the model. In the first iteration of the model, all weights are initialized at random. Combining this with an aggressive loss function like WBCE can cause quite a bit of instability. For every false positive prediction, the model weights produce high loss and receive large corrections. This does not improve the model, but pushes its weights into a new, random and unimproved direction. Phenomena like these can be tracked via the training loss curve, as it will result in sudden erratic spikes, rather than a smooth convergence. To counter this effect, a lower learning rate than originally desired was chosen. The learning rate scales the magnitude of corrections made to model weights and by lowering it, stability is enforced in the beginning of training. A drawback of this is training speed. As the learning rate is lower, more epochs will have to be run to achieve the same loss improvement. Additionally adjusting the learning rate scheduler to spend 30% of the epochs in a warm-up period with further reduced learning rate was a great asset.

To establish the feasibility of our model, Figure 5.1, shows a few sample predictions of Portugal, with varying timeframes. Here, we predicted over the course of an entire calendar year (2025) and show excerpts for each season that can be qualitatively discussed.

5.1. Experiment 1: The Baseline Model

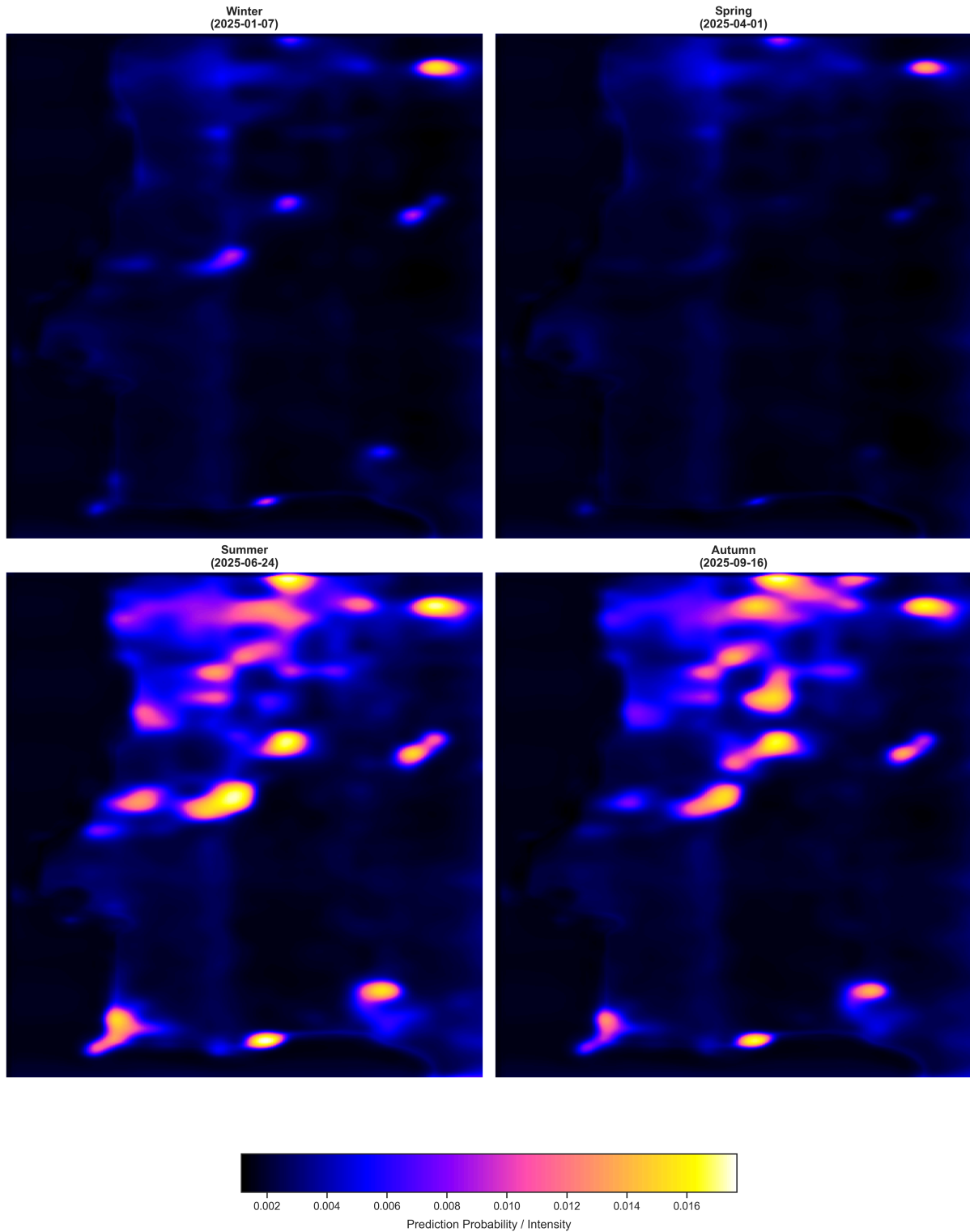


Figure 5.1: Seasonal wildfire probability predictions over Portugal

First, it is highly notable that by exposing the transformer to binary classes (fire/no-fire) and inferring a burn probability from them is possible. By allowing the WBCE loss function to put great emphasis on not missing fires, the model is able to infer similar risk for regions of similar conditions even if no fire was spotted there. This proves that the model operates as a bridge using binary classes to infer probabilities. This allows for outputs to hold much richer information about high risk regions which use inferred information from other pixels.

Next, it is without surprise that the model has successfully learned the difference between land and water, as clearly visible by the values drawing a clear coastline in Figure 5.1. By making concentrated choices with no-data values and masking operations, the attention heads in the backbone seem to know not to infer information into watermass, essentially treating it as a natural barrier to wildfire.

Furthermore, a strong seasonal trend can be detected. Apart from a few anomalies, the overall risk is near zero for early months as Portugal is in winter season. Peak fire periods are June to September as shown, after which rain season in October usually takes wildfire risk down to zero again. By encoding of the Julian day, the general disparity between winter and summer climate or a combination of both, the model has learned the ability to distinguish seasonality.

5.2. Experiment 2: Sample Extent

The first sub-investigation as defined by **RQ 2** in Figure 2.3. Conceptually, this investigation attempts to discover the amount of context necessary for a valuable prediction to be made, by varying the extent of the input sample. The setup of the spatio-transformer allows for dynamic scaling of input features. The temporal scope of dynamic features, or simply the amount of images ingested per data module is variable. However, the early fusion concept chosen here demands every input token to be treated equally. This has the effect that the number of images in the input sample linearly scales the amount of tokens, which in turn scale the computational time quadratically. The maximum sample length feasible on hardware available is seven days, above which could not be investigated. In this experiment a lead-time of zero days was chosen. Four otherwise equally defined models were trained with varying sample extents, the experiment descriptions of which are shown in Table 4.

Experiment Name	Sample Extent
SampleExt1	1
SampleExt3	3
SampleExt5	5
SampleExt7	7

Table 4: Sample extent experiment configurations

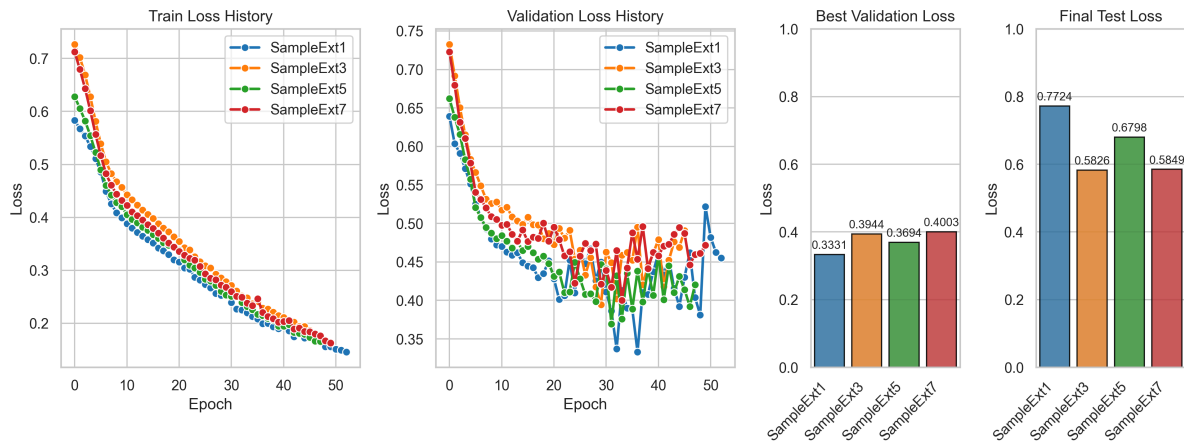


Figure 5.2: Training performance of models with varying sample length

Figure 5.2 shows the training performance of the four respective cases to be compared. Training loss histories show consistent convergence for all cases. Crucial for all of these is the region

5.2. Experiment 2: Sample Extent

in the training loss after which the steepest slope is overcome. At this stage, models have learnt some basic feature relationships and now concretize them. It is at this stage in which the dropout provides the most value. By sporadically zeroing out weights between attending features, the model is forced to rely on different feature relationships to reduce its losses.

All examples experience volatility in validation loss starting at epochs 20-30, while the training loss decreases steadily. This is a clear giveaway for overfitting, where the models are learning to memorize data rather than wildfire relationships.

Perhaps most critically, all runs show gaps between validation and testing scores. This can point to the general disparity of validation and testing sets, however by comparing these differences amongst the models brings some insight into their capabilities. *SampleExt1* shows the biggest gap. More specifically, while obtaining the lowest best validation, it has the worst testing score. Being able to obtain non-complex seems to be feasible in the short-term but fails when data is completely unknown. As wildfires are highly dynamic and instationary phenomena, this shows that such a short sample time frame is insufficient.

On the other end of the spectrum, *SampleExt7* uses the most contextual information to make predictions. While comparably having a higher average validation loss, it has the smallest disparity to its training loss, making it the best generalizer among the four. Richer, more contextualized information seems to have positive impact on both the prediction accuracy as well as the deployability of the model to previously unseen data. *SampleExt3* shows quite similar conditions.

SampleExt5 acts as middle ground to the aforementioned models. Both sit in the middle for validation scores but fail to capture the enhanced generalization of the seven days of context.

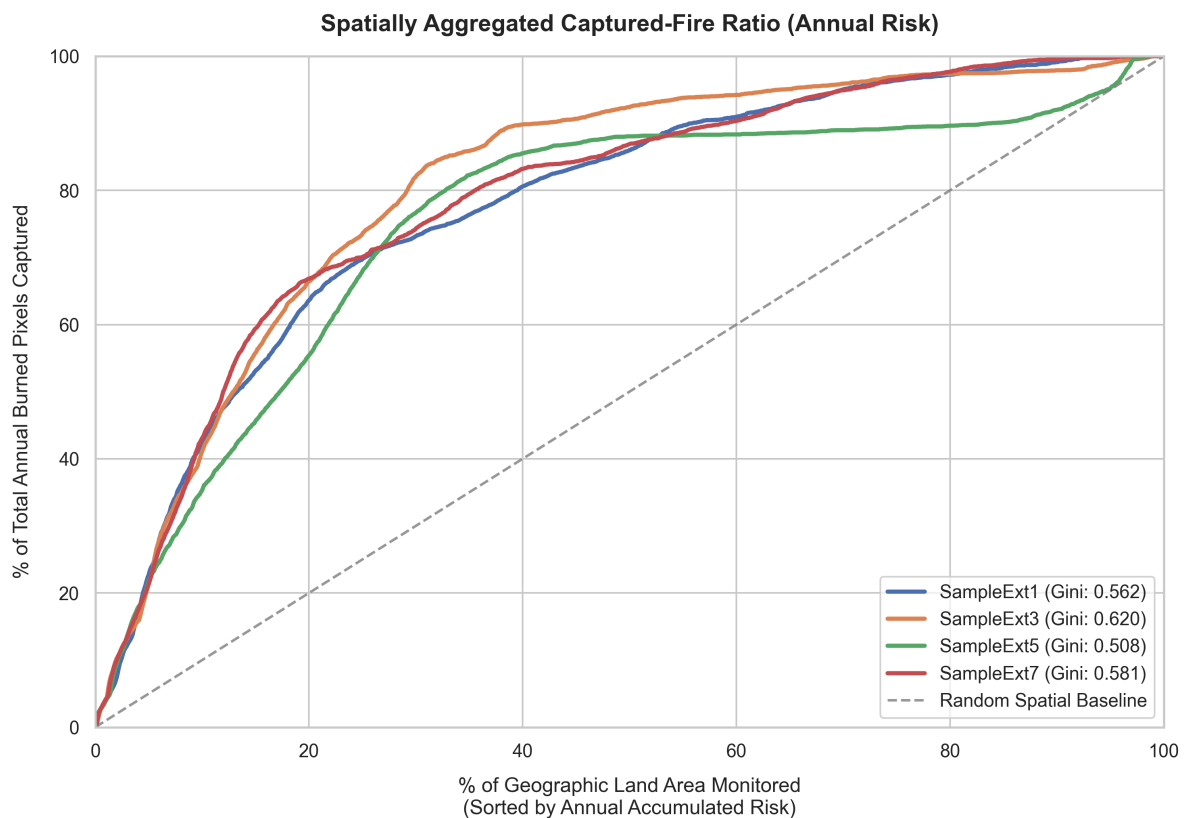


Figure 5.3: Lift curves and Gini-coefficients of comparable models

To further demonstrate the models abilities and illustrate their differences Figure 5.3 illustrates cumulative gains of lift-charts of the models. Essentially, the methodology here argues that for such a rare event like wildfires, most of the risk should be contained in a fraction of the total area. The set of predictions, presented on the x-axis, has here been ordered in descending order of predicted risk. A model that would simply guess randomly would show a uniform distribution of risk over all pixels, as denoted by the random spatial baseline. A well-trained model on the other hand should be able to contain large amounts of the actual fires in very few image pixels.

All four models show this behaviour, proving they operate without having to guess randomly. The clear categorical winner from this perspective is again *SampleExt3*, with the highest overall Gini-coefficient of 0.620. It is able to contain roughly 70% of actual fires within 20% of the total area.

As previously *SampleExt5* underperforms compared to all other models, being able to centralize significantly less burn pixels its riskiest 20% of its area, aligning with the previous loss analysis.

The results of this experiment shows, that context has immense influence on predictive validity. However, simply by adding more context and increasing sample length, accuracy will not always improve. Extremely short one-day samples obtain highest accuracy in finding patterns in known data, however struggle to generalize to unseen samples, as shown by the testing losses. It is with less input features, that a transformer is able to specifically seek out inherent data noise and learn from it to overfit known training samples. On the other end of the spectrum lie the overcontextualized examples. As shown by the training performance and lift-curve of the longest sample length, this maximum context example no longer has the learn trends as well as some the shorter examples. By introducing too much context, the model struggles to relate the amount of information together in a sensible way. This motivates for the existence of a point of diminishing returns. Limiting sample length to three days, provides only the most useful context and while still retaining some inaccuracies as shown by the lift-curve in Figure 5.3 produces the best overall performance.

5.3. Experiment 3: Lead-Time

The second experiment relates to **RQ 3** of Figure 2.3 and investigates the extent to which varying the lead-time influences prediction abilities of the transformer. To reiterate, the lead-time describes the amount of time passed between the last seen training sample and the beginning of the target, the time frame of prediction as per Figure 4.1. In this instance we test for five different lead times, as shown in Table 5. The correct interpretation of *Lead-5* is that the sample and target extents overlap by five days. It therefore does not function as a predictor, but more as a wildfire tracker and has here been included to show the influence of a temporal overlap or a negative lead time.

Experiment Name	Lead-Time
Lead-5	-5
Lead0	0
Lead5	5
Lead10	10
Lead20	20

Table 5: Lead-time experiment configurations

5.3. Experiment 3: Lead-Time

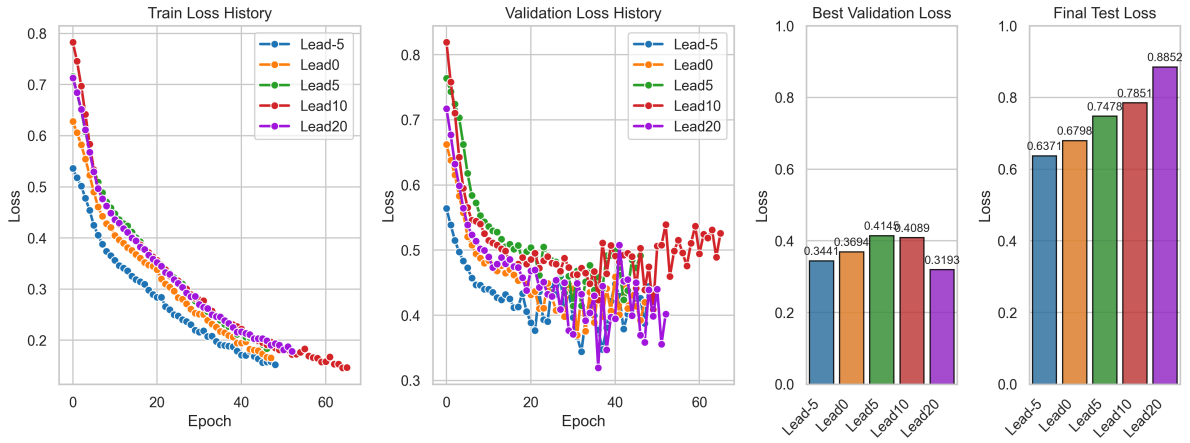


Figure 5.4: Training performance of models with varying lead time

The training performance of all models is useful once again to determine important trends and comparisons between them. Notably and without surprise, the training losses of *Lead-5* are lower and steeper as compared to its counterparts. Nonetheless, despite increasing the lead-times to values as large as 20, both training and validation scores seem to be converge, showing learned trends.

Regardless, of the lead-time, all validation losses become unstable as late as epoch 30, showing first signs of overfitting. All models show some ability to learn fundamental relationships in the features. Even after the point of the best validation loss is reached even the longest lead time model *Lead-20* shows the ability to overfit and predict targets that take place 20 days into the future.

Most interesting in this case is the testing loss chart, which shows a staircase behaviour, when elongating the lead-time. This shows a very clear pattern: the longer the lead-time, the more predictive accuracy degrades for unseen data. Despite its validation loss being exceptional (0.3193), *Lead-20* far overfits the data as can be seen by its test loss value (0.8852).

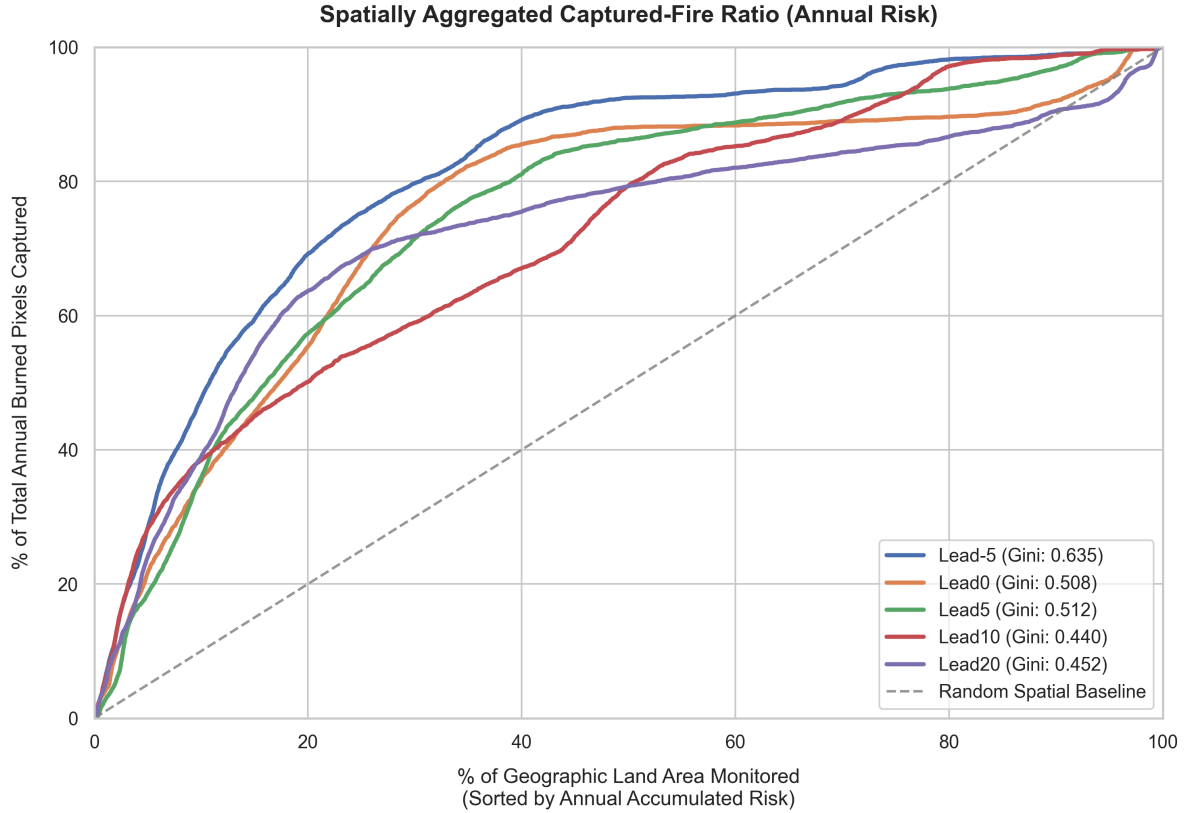


Figure 5.5: Lift curves and Gini-coefficients of comparable models

The lift curve shows similar trends. With a Gini-coefficient of 0.635 sets the baseline for all future predicting models. Unsurprisingly both *Lead20* (Gini: 0.452) and *Lead10* (Gini: 0.440) perform the worst. Surprisingly however, *Lead5* (Gini : 0.512) shows little to none prediction degradation compared to *Lead0* (Gini : 0.508), the baseline model, despite its extended target frame of reference. Predictions show to retain their predictive validity for up to a five-day period, however not for far greater lead-times.

Combining insights from both analyses shows, that a five-day lead-time is highly performant under predictive circumstances. Both a stable Gini-coefficient of the lift curve and relatively good validation to testing loss ratio can be seen for this model making it a viable option for fire management purposes. Furthermore, *Lead-5* underlines the validity of the architecture, by showing that it can spatially segment features. Long lead-times here stress test the network showing that inherent noise and missing data are issues that are especially extended into the future prediction domain. Nonetheless, the ability of the transformer to retain predictive performance without losing copious amounts of generalizability demonstrates that an applicable predictive lead-time is possible in this transformer architecture.

5.4. Experiment 4: Ablation Studies

Frequently, due to the complex nature of transformer architectures, not enough reasoning about the inner workings and performance is done (Figure 2.3, RQ4). Hence, we here devote a section to ablation studies, more specifically to test the performance of the model under constrained circumstances.

To offer a structural window into the behaviour of the model we here take a trained model and predict one year of data with one or multiple input features removed. For each, we track

5.4. Experiment 4: Ablation Studies

both the mean absolute error (MAE) and the mean squared error (MSE), the ladder penalizing especially large errors. All comparisons are made to the pre-trained baseline model *SampleExt3* with a lead-time of zero, sequence extent of three and a target prediction of 14 days. Computationally, this is achieved by intervening in the forward pass of the prediction. By setting input data of single or combinations of models to zero, all attention blocks which rely on this information will yield zero as well. In essence, this constraint allows the model to behave as if certain information (e.g. terrain) is not available.

Other ablation approaches include retraining the model entirely under these constrained circumstances, however it is highly probable that the models will find new shortcuts to circumvent their limitations. This method would furthermore focus on investigating the data redundancy rather than evaluating the focus of the architecture, the ladder being the goal of this investigation. By removing single data modules we investigate the models dependencies and structural vulnerabilities.

In total we here test 12 different scenarios, which include single or combinations of input data modules:

Ablation Configurations	Number of Modules
Aspect	2
Slope	1
Aspect + Slope	3
Roads Proximity	1
Burn History	1
Precipitation	1
NDVI	1
NDWI	1
Wind Direction	2
Wind Speed	1
Wind Direction + Speed	3
LST	1

Table 6: Ablation configurations investigated for single and combinations of data modules

The global sensitivity analysis by data showing the ablation results for all 12 configurations is shown in Figure 5.7. The most important confirmation here is one that as was expected from discussions in Section 2, terrain is a main driver of predictional robustness. Removing the combined aspect and slope modules (MAE: 0.07080, MSE: 0.01336) removes all systematic overview of the model, making it the single most important metric of the model. Another notable result is that it by removing just one of the two terrain types (aspect or slope), the error is somewhat contained. Both modularities appear to be appear an error safeguard for the respective other, as described by their comparably small MSE (Aspect: 0.00102, Slope: 0.00029). The dominating feature under the dynamic features is LST (MAE: 0.05223, MSE: 0.00945). It acts as the anchor of thermal context, naturally scaling wildfire probabilities.

Roads (MSE: 0.03627), burn history (MSE: 0.03258) and wind components (MSE: 0.02941) represent the second tier of feature importance. They don't carry copious amounts weight, but in combination provide the model with critical boundary logic. While both burn history and

road proximity are not dynamic features, but aid the prediction crucial secondary information regarding proximity to civilization.

NDVI (MSE: 0.00973) is only about half as impactful as NDWI (MSE: 0.01987). Given their similar derivations from satellite imagery, the two correlate quite closely, however plant moisture stress (NDWI) appears far more meaningful than proxies for photosynthetic capacity as NDVI.

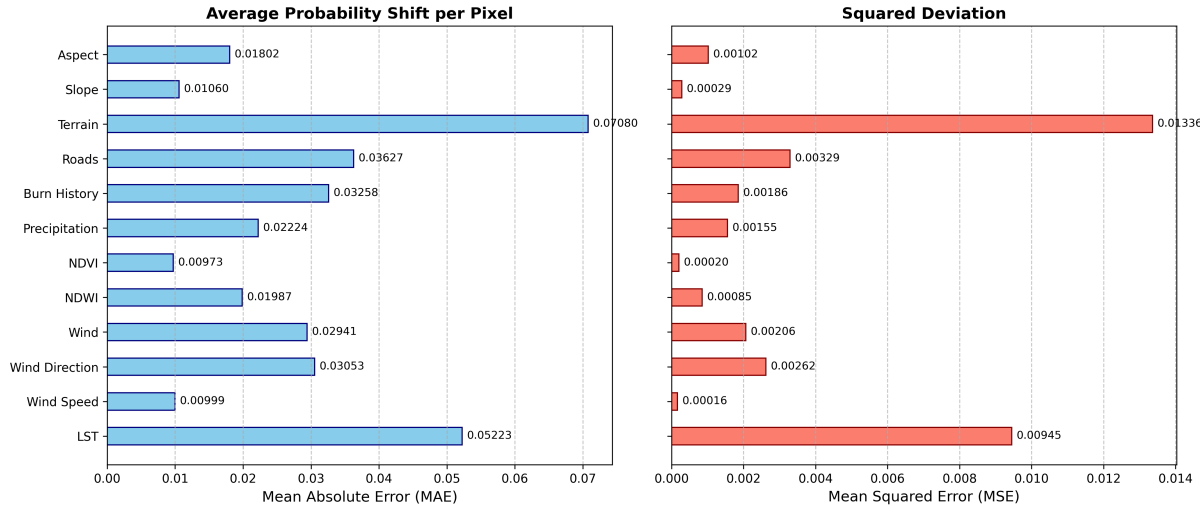


Figure 5.6: Model Sensitivity Analysis by Data Module

To show not just the overall importance of features, this investigation additionally tracked feature importance over the period of an entire year, for equal periods of 14 days. In order to highlight seasonal impact of modules, here illustrated is a temporal sensitivity analysis (Figure 5.7).

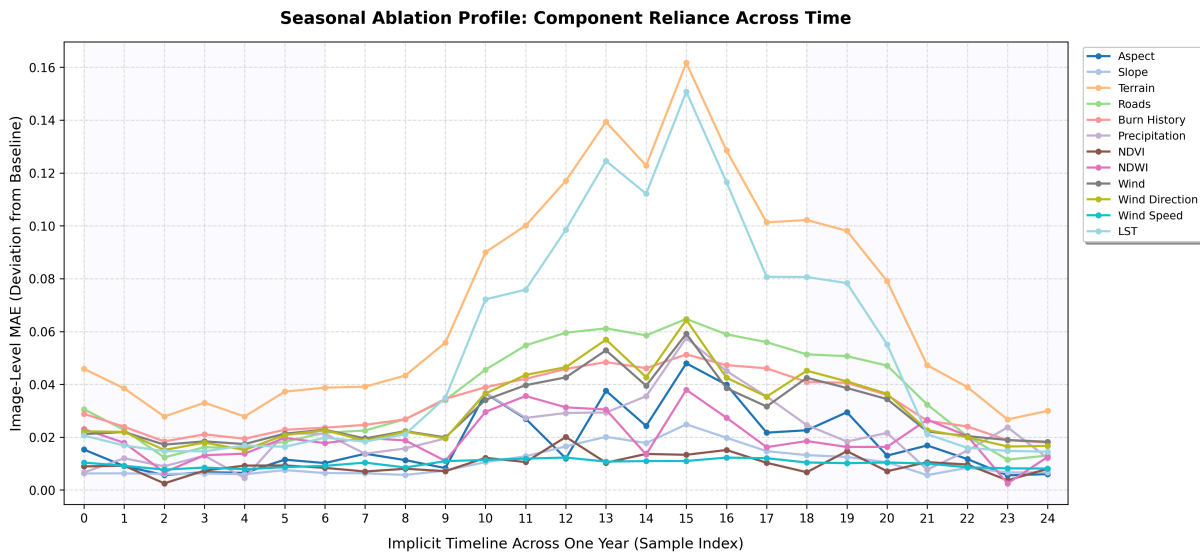


Figure 5.7: Seasonal Ablation Profile: Component Reliance Across Time

The key notion shown in Figure 5.7 is that the induced error from removing modalities is variable by season. For most ablations, the model prediction error remains stable for the first nine indices, or about four months. This is the probable time for wildfire occurrence, hence none of the conditions in any module are fulfilled to cause significant risk. Quite clearly, since the model is predicting imbalanced binary classes, it requires less information to predict the

5.4. Experiment 4: Ablation Studies

more likely class as more information exists for it. For all ablations considered, MAE shifts rapidly as soon as wildfires are in season, at around indices 9-10. It is at this point, that the structural reasoning changes from relying on the broadness of all modules in winter, to predicting higher probabilities from a few extreme values in the summer months. Perhaps most notably, removing terrain features and LST causes significant errors of upto 16% in peak season. Furthermore, LST and terrain features show correlating error behaviour in their peaks from indices 14-16, stating that during this peak season, the features interact with one another.

Features contributing significantly less error especially in the seasonal perspective are NDVI and wind speed despite both of these being large datasets producing significant amounts of input tokens. Both stay almost perfectly level, causing little inaccuracies to the model when omitted. Rather than being heavily reliant on them, the model treats these features as a baseline offset and does not adjust them by season.

An important take-away from this ablation study, especially when evaluating RQ4, is that evaluating feature importance is entirely possible. By showing a distributed feature reliance, it is clear that the model does not simply use shortcut learning, depending on a single variable to produce as little less as possible, but uses the entire arsenal of its data types. Figure 5.6 showed that features do have relative levels of impact on the prediction outcome, while none exceed an error of 7%. This shows that the cross-attention backbone of the architecture is successfully using information from all modalities. Another important argument to be made, is that despite dealing with immense cloud-cover in the winter months, all dynamic weather-dependent data types suppress this noise, producing no larger error than the static features. The model is however to some extent sensitive to imbalance of classes, which can be detected from rising error for all module ablations in fire-prone months.

6. Conclusions

Finalization
in progress,
left blank
here

Bibliography

- Aldersley, A., Murray, S. J., & Cornell, S. E. (2011). Global and regional analysis of climate and human drivers of wildfire. *Science of the Total Environment*, 409(18), 3472–3481. <https://doi.org/https://doi.org/10.1016/j.scitotenv.2011.05.032>
- Andrianarivony, H. (2024). *Fire*, 7(12), 482. <https://doi.org/10.3390/fire7120482>
- Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lucic, M., & Schmid, C. (2021). ViViT: A Video Vision Transformer. *Corr.* <https://arxiv.org/abs/2103.15691>
- Arndt, N., Vacik, H., Koch, V., Arpaci, A., & Gossow, H. (2013). Modeling human-caused forest fire ignition for assessing forest fire danger in Austria. *Iforest - Biogeosciences and Forestry*, (6), 315–325. <https://doi.org/10.3832/for0936-006>
- Babrauskas, V. (2025). The Palisades Fire of Los Angeles: Lessons to Be Learned. *Fire*. <https://doi.org/10.3390/fire8080303>
- Bazi, Y., Bashmal, L., Al Rahhal, M., Dayil, R., & Ajlan, N. (2021). Vision Transformers for Remote Sensing Image Classification. *Remote Sensing*, 13, 516. <https://doi.org/10.3390/rs13030516>
- Castro, F. M. C., Silva, J. S., Fernandes, P., & Rigolot, E. (2010). Towards integrated fire management: outcomes of the European project Fire Paradox. *European Forest Institute*, 23.
- Chen, B., & Jin, Y. (2022). Spatial patterns and drivers for wildfire ignitions in California. *Environmental Research Letters*, 17(5). <https://doi.org/10.1088/1748-9326/ac60da>
- Dong, L., Wang, Y., Li, C., Zhu, W., Yu, H., & Tian, H. (2025). A Joint Transformer–XGBoost Model for Satellite Fire Detection in Yunnan. *Fire*, 8(10). <https://doi.org/10.3390/fire8100376>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *Corr.* <https://arxiv.org/abs/2010.11929>
- Dubey, P., Rakesh, N., Dubey, P., & Bokoro, P. N. (2025). From Convolution to Attention: Transformer-Based Modeling for Multi-Day Wildfire Spread Forecasting. *IEEE Access*, 13, 128895–128919. <https://doi.org/10.1109/ACCESS.2025.3586922>
- Durlević, U., Ilić, V., & Valjarević, A. (2025). Wildfire Susceptibility Mapping Using Deep Learning and Machine Learning Models Based on Multi-Sensor Satellite Data Fusion: A Case Study of Serbia. *Fire*, 8(10). <https://doi.org/10.3390/fire8100407>
- Gao, Z., Shi, X., Hao Wang, Y. Z., Wang, Y., Li, M., & Yeung, D.-Y. (2022). Earthformer: Exploring Space-Time Transformers for Earth System Forecasting. *Neurips*. <https://doi.org/10.48550/arXiv.2207.05833>
- Heo, B., Park, S., Han, D., & Yun, S. (2024). Rotary Position Embedding for Vision Transformer. <https://arxiv.org/abs/2403.13298>

- Hochreiter, S. (1998). The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2), 107–116. <https://doi.org/10.1142/S0218488598000094>
- Illarionova, S., Shadrin, D., Tregubova, P., Ignatiev, V., Efimov, A., Oseledets, I., & Burnaev, E. (2022). A Survey of Computer Vision Techniques for Forest Characterization and Carbon Monitoring Tasks. *Remote Sensing*, 14(22). <https://doi.org/10.3390/rs14225861>
- Jain, P., Coogan, S. C., Subramanian, S. G., Crowley, M., Taylor, S., & Flannigan, M. D. (2020). A review of machine learning applications in wildfire science and management. *Environmental Reviews*, 28(4), 478–505. <https://doi.org/10.1139/er-2020-0019>
- Kelley, D. I. (2025). *State of Wildfires 2024-2025*. <https://doi.org/10.5194/essd-17-5377>
- Khan, A., Rauf, Z., Sohail, A., Khan, A. R., Asif, H., Asif, A., & Farooq, U. (2023). A survey of the vision transformers and their CNN-transformer based variants. *Artificial Intelligence Review*, 56(S3), 2917–2970. <https://doi.org/10.1007/s10462-023-10595-0>
- Kingma, D. P. (2014). Adam: A Method for Stochastic Optimization. *Conference Paper at ICLR 2015*. <https://arxiv.org/abs/1412.6980>
- Li, B. S., & Rad, R. (2024). Wildfire Spread Prediction in North America Using Satellite Imagery and Vision Transformer. *2024 IEEE Conference on Artificial Intelligence (CAI)*, 1536–1541. <https://doi.org/10.1109/CAI59869.2024.00278>
- Li, B. (2021). Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *Elsevier*. <https://doi.org/10.1016/j.ijforecast.2021.03.012>
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021,). *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. <https://arxiv.org/abs/2103.14030>
- Luo, X., Nadiga, B. T., Park, J. H., Ren, Y., Xu, W., & Yoo, S. (2022). A Bayesian Deep Learning Approach to Near-Term Climate Prediction. *Journal of Advances in Modeling Earth Systems*, 14(10), e2022MS003058. <https://doi.org/10.1029/2022MS003058> e2022MS003058
- Meijer, J. R., Huijbregts, M. A. J., Schotten, K. C. G. J., & Schipper, A. M. (2018). Global patterns of current and future road infrastructure. *Environment Research Letters*, 13(6). <https://doi.org/10.1088/1748-9326/aabd42>
- Naeve, Z., Mitchell, L., Reed, C., Campbell, P., Morgan, T., & Rogers, V. (2024). Introducing Dynamic Token Embedding Sampling of Large Language Models for Improved Inference Accuracy. *Techrxiv*, 2024(1028), . <https://doi.org/10.36227/techrxiv.173014793.37761346/v1>
- Papa, L., Russo, P., Amerini, I., & Zhou, L. (2024). A Survey on Efficient Vision Transformers: Algorithms, Techniques, and Performance Benchmarking. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12), 7682–7700. <https://doi.org/10.1109/tpami.2024.3392941>
- Pytorch-Contributors. (2019b). *torch.optim*. pytorch. <https://docs.pytorch.org/docs/2.11/optim.html>
- Pytorch-Contributors. (2019a). *torch.tensor*. pytorch. <https://docs.pytorch.wiki/en/tensors.html?>

- Rafaqat, W., Iqbal, M., Kanwal, R., & Song, W. (2022). Study of Driving Factors Using Machine Learning to Determine the Effect of Topography, Climate, and Fuel on Wildfire in Pakistan. *Remote Sensing*, 14, 28. <https://doi.org/10.3390/rs14081918>
- Ridnik, T., Ben-Baruch, E., Noy, A., & Zelnik-Manor, L. (2021,). *ImageNet-21K Pretraining for the Masses*. <https://arxiv.org/abs/2104.10972>
- Rothermel, R. C. (1972). *A mathematical model for predicting fire spread in wildland fuels*. <https://research.fs.usda.gov/treesearch/32533>
- Sanderson, G. (2024b). *Attention in transformers, step-by-step*. <https://www.3blue1brown.com/lessons/attention>
- Sanderson, G. (2024a). *Transformers, the tech behind LLMs*. <https://www.3blue1brown.com/lessons/gpt/#transformers-the-tech-behind-llms-deep-learning-chapter-5>
- Shaw, P., Uszkoreit, J., & Vaswani, A. (2018). Self-Attention with Relative Position Representations. *Corr.* <http://arxiv.org/abs/1803.02155>
- Syphard, A. D., Keeley, J. E., Gough, M., Lazarz, M., & Rogan, J. (2022). What Makes Wildfires Destructive in California?. *Fire*, 5(5). <https://doi.org/10.3390/fire5050133>
- Vaswani, A. (2017). *Attention Is all You Need*. <https://doi.org/10.48550/arXiv.1706.03762>
- Wang, R., Ma, L., & He, G. (2024). *Transformers for Remote Sensing: A Systematic Review and Analysis*. *Sensors (Basel)*. <https://doi.org/10.3390/s24113495>
- Wang, Y., Deng, Y., Zheng, Y., Chattopadhyay, P., & Wang, L. (2025). Vision Transformers for Image Classification: A Comparative Survey. *Technologies*, 13(1). <https://doi.org/10.3390/technologies13010032>
- Westerling, A. L., Hidalgo, H. G., Cayan, D. R., & Swetnam, T. W. (2006). Warming and Earlier Spring Increase Western U.S. Forest Wildfire Activity. *Science*, 313(5789), 940–943. <https://doi.org/10.1126/science.1128834>
- Xu, D., Ruan, C., Kumar, S., Körpeoglu, E., & Achan, K. (2019). Self-attention with Functional Time Representation Learning. *Corr.* <http://arxiv.org/abs/1911.12864>
- Xu, Z., Cheng, S., Wang, L., He, H., Sun, W., Li, J., & Xu, L. L. (2026). BCWildfire: A Long-term Multi-factor Dataset and Deep Learning Benchmark for Boreal Wildfire Risk Prediction. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(46), 39486–39494. <https://doi.org/10.1609/aaai.v40i46.41299>
- Álvarez-Díaz, M., González-Gómez, M., & Otero-Giraldez, M. S. (2015). Detecting the socioeconomic driving forces of the fire catastrophe in NW Spain. *European Journal of Forest Research*, 123, 1087–1094. <https://doi.org/10.1007/s10342-015-0911-1>

